CIU9872B_01(1.1)_EXT
芯片函数库用户使用手册

版本 V1.2

仅用于东信开发使用



北京中电华大电子设计有限责任公司
CEC Huada Electronic Design Co.,Ltd

2026 年 4 月

声　明

本手册的版权属北京中电华大电子设计有限责任公司所有。任何未经授权对本手册进行复印、印刷、出版发行的行为，都将被视为是对北京中电华大电子设计有限责任公司版权的侵害。北京中电华大电子设计有限责任公司保留对此行为诉诸法律的权力。

北京中电华大电子设计有限责任公司保留未经通知用户对本手册内容进行修改的权利。虽然我们已经核对本手册的内容，但是差错有时候难以完全避免，所以，我们会对手册的内容进行定期的审查，并在下一版的文件中作必要的修改。建议您在最终设计前从华大电子获取本文档的最新版本。

仅用于东信开发使用

变更记录

版本	修改描述	日期
V1.0	初稿	2024-11-08
V0.2	删除文档中 SWP 接口描述，但保留 CRC 对 SWP 校验模式的计算功能	2025-01-10
V0.3	修改 HED_ECCDecrypt_SEC 函数中解密失败返回值说明错误的问题	2025-04-08
V0.4	增加附录 B，NVM 擦写时间	2025-10-27
V1.0	升级版本为 V1.0	2025-11-22
V1.1	修改 7816 API 接口、DMA 接口	2026-03-09
V1.2	删除 PokerCheck 接口	2026-04-01

仅用于东信开发使用

引　言

[编程指南介绍]

本文档描述 CIU9872B_01_V1.1_EXT 产品的 API 函数，提供给客户用于嵌入式软件开发。

[编写目的]

本文档适用于 COS/Java 平台用户，介绍或说明 API 的功能/接口定义/用法等。

[各章节内容介绍]

1　概述

介绍芯片实现那些 API 功能。

2　用户区的 API

介绍与用户 COS 一起编译的 API 功能/接口定义/用法等。

3　芯片算法 API 区的 API

介绍芯片算法 API 区的 API 功能/接口定义/用法等。

4　芯片 NVM API 区的 API

介绍芯片 NVM API 区的 API 功能/接口定义/用法等。

5　附录 A

介绍算法参数 type 定义

仅用广东信开发使用

目录

1 概述 1
2 用户区 API 1
2.1 概述 1
2.2 HAL 层 API 1
2.2.1 HAL 层 API 通用结构定义 2
2.2.2 7816 接口 API 2
2.2.3 DMA 接口 API 4
2.2.4 GPIO 接口 API 6
2.2.5 IO 控制类 API 9
2.2.6 其他类 API 10
2.3 增强芯片稳定性方案 API 16
2.3.1 概述 16
2.3.2 函数接口描述 16
2.4 随机数类 API 19
2.4.1 概述 19
2.4.2 函数接口描述 19
2.5 对称算法 GCM 模式 API 21
2.5.1 功能介绍 21
2.5.2 数据结构 21
2.5.3 函数接口描述 22
2.6 对称算法 CCM 模式 API 25
2.6.1 功能介绍 25
2.6.2 数据结构 26
2.6.3 函数接口描述 27
3 芯片掩膜算法 API 31
3.1 算法类 API 31
3.1.1 概述 31
3.1.2 函数接口描述 31
3.2 其他类 API 58
3.2.1 概述 58
3.2.2 函数接口描述 58
4 芯片掩膜 NVM API 63

仅用于东信开发使用

4.1 概述 63
4.1.1 NVM 用户区分区介绍 63
4.1.2 NVM API 的主要特征 63
4.1.3 Flash 增强稳定性功能 63
4.1.4 NVM API 注意事项 64
4.2 HEFlash 区擦写函数 64
4.2.1 初始化 HEFlash 区函数 64
4.2.2 HEFlash 擦写函数 65
4.2.3 特殊场景高可靠性擦写函数 66
4.3 Common Flash 区擦写函数 69
4.3.1 功能介绍 69
4.3.2 接口定义 69
4.4 Common Flash 区页擦函数 69
4.4.1 功能介绍 69
4.4.2 接口定义 69
4.5 Common Flash 区字写函数 70
4.5.1 功能介绍 70
4.5.2 接口定义 70
4.6 用户参数区 Flash 擦写函数 70
4.6.1 功能介绍 70
4.6.2 接口定义 71
4.7 读 NVMAPI 版本函数 71
4.7.1 功能介绍 71
4.7.2 接口定义 71
5 附录 A 对称算法 Type 定义 72
6 附录 B NVM 擦写时间 72

仅用于东信开发使用

1 概述

CIU9872B_01(1.1)_EXT 芯片的 API 分两种提供方式：

- 一种是用户区 API（与用户 COS 一起编译的源码或 LIB 形式的 API），详情请参见第 2 章；
- 一种是掩膜类 API（掩膜在芯片 API 区，不占用户 NVM 空间），包括算法 API 及 NVM（Flash）擦写函数。详情请参考第 3 章与第 4 章。

2 用户区 API

2.1 概述

用户区 API 采用分层设计，分为 Application 层、Middleware 层、以及 HAL（全称 Hard Abstraction Lay）层，分层结构见下图：

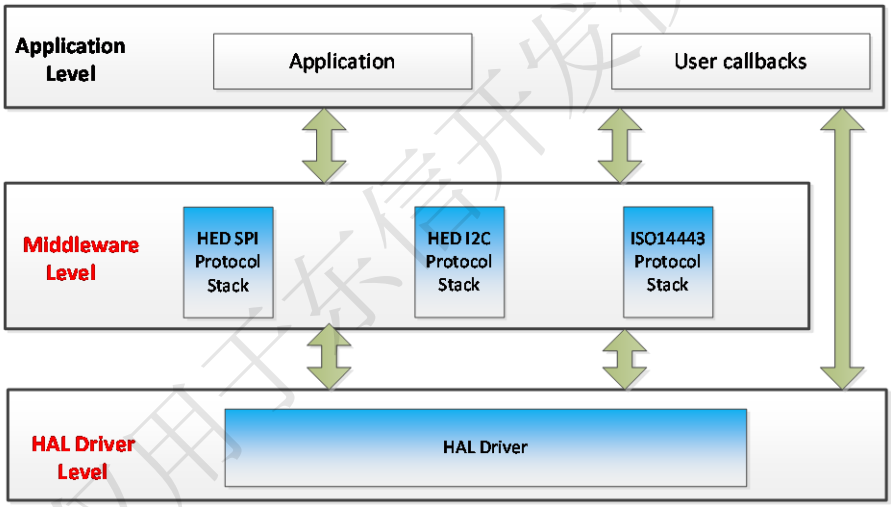


图 2-1 用户区 API 分层结构示意图

本次产品包中用户区 API 主要分为通讯类/DMA 类/IO 控制类/GPIO 类/随机数类/增强芯片稳定性方案类/版本管理类等。

各类模块均提供 HAL Driver 类 API，通讯类 API 额外提供 Middleware 层的 ISO 14443 协议栈、HED SPI/I2C 协议栈（遵循《HED SPI/I2C 通讯规范》）；

上图中 Middleware 层可以没有，即直接通过 Application 层调用 HAL 层 API。

Application 层一般由用户实现。

2.2 HAL 层 API

HAL 层 API 主要为 7816、SPI、I2C、14443、DMA、IOControl、GPIO 以及一般功能类 HAL API。

仅用于华虹开发使用

2.2.1 HAL 层 API 通用结构定义

HAL 层 API 通用结构体定义如下：

API 函数返回类型结构定义如下：

```
typedef enum
{
    HAL_OK          = 0x00U,    //返回成功
    HAL_ERROR  = 0x01U,    //返回失败
    HAL_BUSY        = 0x02U,    //线程 IP 模块被占用
    HAL_TIMEOUT     = 0x03U         //等待超时
} HAL_StatusTypeDef;
```

API 函数锁定类型定义

```
typedef enum
{
    HAL_UNLOCKED    = 0x00U,    //线程未被锁定
    HAL_LOCKED      = 0x01U         //线程已锁定
} HAL_LockTypeDef;
```

2.2.2 7816 接口 API

2.2.2.1 查询方式发送多字节数据 API

2.2.2.1.1 功能介绍

7816 接口数据发送。

2.2.2.1.2 接口定义

函数接口	unsigned int HED_T0_SendNBytes (unsigned char *DataAddr, unsigned int DataLen)
输入	➢ DataAddr：待发送数据的起始地址 ➢ DataLen：待发送数据的字节长度
输出	无
返回值	➢ 0x00000000：表示发送正确 ➢ 0x00000001：表示发送错误 ➢ 0x00000002：发送长度为 0 错误
说明	➢ 如果某字节数据发送错误，该字节最多重发 3 次（即最多发送 4 次）； ➢ API 内不负责打开 SCI 模块门控时钟。

仅用于东信开发使用

2.2.2.2 查询方式发送 1 字节数据 API

2.2.2.2.1 功能介绍

7816 接口 1 字节数据发送。

2.2.2.2.2 接口定义

函数接口	void HED_T0_SendByte (unsigned char i)
输入	➢ i：待发送数据
输出	无
返回值	无
说明	➢ API 内不负责打开 SCI 模块门控时钟。

2.2.2.3 查询方式接收多字节数据到 RAMAPI

2.2.2.3.1 功能介绍

7816 接口数据接收。

2.2.2.3.2 接口定义

函数接口	void HED_T0_ReceiveNBytesRam (unsigned char *DataAddr, unsigned int DataLen)
输入	➢ DataLen：串口接收数据的字节长度
输出	➢ DataAddr：串口接收到 RAM 中数据的起始地址
返回值	无
说明	➢ 函数只提供将数据接收到 RAM 中； ➢ API 内不负责打开 SCI 模块门控时钟。

2.2.2.4 延时 ETU

2.2.2.4.1 功能介绍

ETU 延时接口；

2.2.2.4.2 接口定义

函数接口	void HED_DelayNetu(uint8_t delay)
输入	➢ Delay：ETU 个数
输出	➢ 无
返回值	无

仅用于华信开发使用

说明	无

2.2.2.5 打开自动发送 WTX

2.2.2.5.1 功能介绍

开启硬件自动发送 0x60 机制；

2.2.2.5.2 接口定义

函数接口	void HED_OpenSend60(void)
输入	➢ 无
输出	➢ 无
返回值	无
说明	无

2.2.2.6 关闭自动发送 WTX

2.2.2.6.1 功能介绍

关闭硬件自动发送 0x60 机制；

2.2.2.6.2 接口定义

函数接口	void HED_CloseSend60(void)
输入	➢ 无
输出	➢ 无
返回值	无
说明	无

2.2.3 DMA 接口 API

2.2.3.1 术语定义

无。

2.2.3.2 数据结构定义

■ DMA HAL 层初始化结构体定义

```
typedef struct
{
    unsigned int        Channel;
    unsigned int        ChannelSelect;
```

仅用于芯信开发使用

```
    unsigned int        Priority;
    unsigned int        TransBeatType;
    unsigned int        TransModeType;
    unsigned int        BlockSizeUnit;
    unsigned int        DataWidth;
    unsigned int        Direction;
} DMA_InitTypeDef;
```

■ DMA HAL 层结构体句柄定义

```
typedef struct DMA_HandleTypeDef
{
    DMA_InitTypeDef             Init;
    HAL_LockTypeDef             Lock;
    void (* TransCmpltCallback)(struct DMA_HandleTypeDef *hdma);
    void (* TransErrorCallback)(struct DMA_HandleTypeDef *hdma);
    void        *pUpLevelDataPtr; /*!< point the Upper Level Handle        */
}DMA_HandleTypeDef ;
```

2.2.3.3 函数接口描述

本 DMA 类 API 主要提供 DMA 初始化，DMA 复位，DMA 传输相关 API。

2.2.3.3.1 DMA 初始化 API

2.2.3.3.1.1 功能介绍

根据用户配置的参数初始化 DMA，同时 API 内部会自动开启 DMA 时钟。

2.2.3.3.1.2 接口定义

函数接口	HAL_StatusTypeDef HAL_DMA_Init(DMA_HandleTypeDef *hDMA)
输入	➢ hDMA：指向 HAL 层 DMA 结构体句柄的指针
输出	无
返回值	➢ HAL_StatusTypeDef 定义值
说明	➢ 函数内部会开启 DMA 时钟； ➢ 如果在外设结构中使用了 DMA 结构的引用，则需要关联 DMA 与外设的关系； ➢ 对 hDMA->Init 进行初始化配置操作（即对 DMA_HandleTypeDef 结构内的 DMA_InitTypeDef 结构进行初始化配置）。

仅用于东信开发使用

2.2.3.3.2 DMA 复位 API

2.2.3.3.2.1 功能介绍

与 HAL_DMA_Init 操作相反，执行此 API 接口会复位 DMA 配置，会关闭 DMA 时钟。

2.2.3.3.2.2 接口定义

函数接口	HAL_StatusTypeDef HAL_DMA_DeInit(DMA_HandleTypeDef *hDMA)
输入	➢ hDMA：指向 HAL 层 DMA 结构体句柄的指针
输出	无
返回值	➢ HAL_StatusTypeDef 定义值
说明	➢ 函数内部会关闭 DMA 时钟。

2.2.3.3.3 DMA 启动传输 API

2.2.3.3.3.1 功能介绍

此 API 接口为启动 DMA 传输，需要在 DMA Init API 后使用。

2.2.3.3.3.2 接口定义

函数接口	HAL_StatusTypeDef HAL_DMA_Start(DMA_HandleTypeDef *hDMA, unsigned int DstAddr, unsigned int SrcAddr, unsigned int BlockCount)
输入	➢ hDMA：DMA_HandleTypeDef 结构指针； ➢ DstAddr：目的地地址，可以是外设地址或内存地址，Bit0~31 有效； ➢ SrcAddr：源地址，可以是外设地址或内存地址，Bit0~31 有效； ➢ BlockCount：期望传输的 Block 块数，传输的总长度 DataLen=BlockCount×BlockSize（BlockSize 参考 BlockSizeUnit 定义）。
输出	无
返回值	➢ HAL_StatusTypeDef 定义值
说明	➢ 本函数仅配置 DMA 传输目的地址，源地址以及传输长度； ➢ SPI/I2C 等 IP 采用 DMA 传输时，需要在 SPI/I2C 收发函数体中对各 IP DMA 传输进行使能。

2.2.4 GPIO 接口 API

本章节定义了 GPIO 相关操作的 API 接口，可用于 GPIO 模块管脚对数据进行写入或读出操作。

2.2.4.1 术语定义

无。

2.2.4.2 数据结构定义

- PIN 电平状态定义

仅用于客户开发使用

```
typedef enum
{
    GPIO_PIN_RESET      = 0U,
    GPIO_PIN_SET        = 1U
} GPIO_PIN_STATE;
```

- PIN 方向定义

```
typedef enum
{
    GPIO_PIN_DIR_IN         = 0U,
    GPIO_PIN_DIR_OUT        = 1U
} GPIO_PIN_DIR;
```

2.2.4.3 函数接口描述

2.2.4.3.1 GPIO 方向输入/输出 API

2.2.4.3.1.1 功能介绍

设置当前 pin 管脚为输入/输出模式，即设置管脚模式为接收/发送状态。

本项目中 PIN 管脚 GPIO0~GPIO2 与 7816 接口中 RST、CLK 及 IO 复用，除 IO 管脚外，其余管脚不可配置输入/输出模式；GPIO3~GPIO7 可配置输入/输出模式。

2.2.4.3.1.2 接口定义

函数接口	void HAL_GPIO_Direct(unsigned int PinPos, GPIO_PIN_DIR PinDir)
输入	➢ PinPos：PIN 引脚； ➢ PinDir：GPIO_PIN_DIR 类型。
输出	无
返回值	无
说明	➢ 接口调用前需要先执行 IO 配置 API； ➢ 接口定义与 IO 配置 API 功能有重复，目的是在不改变 IO 管脚其它属性的前提下快速对 IO 输入输出模式进行配置，特别是 IO 在复用为单线 IO 模式通讯时，发送与接收可通过此接口设置。

2.2.4.3.2 GPIO 数据写入 API

2.2.4.3.2.1 功能介绍

当 IOCTRL 模块被复用为普通 GPIO 接口使用时，可调用此接口对具体 PIN 进行数值写入操

仅用于东信开发使用

作。

2.2.4.3.2.2 接口定义

函数接口	HAL_StatusTypeDef HAL_GPIO_WritePin(unsigned int PinPos, GPIO_PIN_STATE PinState)
输入	➢ PinPos：PIN 引脚； ➢ PinState：GPIO_PIN_STATE 类型。
输出	无
返回值	➢ HAL_StatusTypeDef 定义值
说明	➢ 函数内部会自动将 PinPos 设置为输出方向； ➢ 接口仅对一条 PIN 管脚值进行写入。

2.2.4.3.3 GPIO 数据读取 API

2.2.4.3.3.1 功能介绍

当 IOCTRL 模块被复用为普通 GPIO 接口使用时，可调用此接口从某一 PIN 上读取数据。

2.2.4.3.3.2 接口定义

函数接口	GPIO_PIN_STATE HAL_GPIO_ReadPin(unsigned int PinPos)
输入	➢ PinPos：PIN 引脚
输出	无
返回值	➢ GPIO_PIN_STATE 定义值
说明	➢ 函数内部会自动将 PinPos 设置为输入方向； ➢ 接口仅对一条 PIN 管脚值进行读取。

2.2.4.3.4 GPIO 中断 API

2.2.4.3.4.1 功能介绍

IOCTRL 模块中断处理函数。

2.2.4.3.4.2 接口定义

函数接口	void HAL_GPIO_IRQHandler(unsigned int PinPos)
输入	➢ PinPos：PIN 引脚
输出	无
返回值	无
说明	➢ 函数内清除当前 PIN 的中断事件

仅用于东信开发使用

2.2.5 IO 控制类 API

本章节定义了 IO 相关操作的 API 接口，可用于 IOCTRL 模块管脚复用、属性配置等操作。当 IO 管脚被复用为 GPIO 功能时，可通过“GPIO 类 API_HAL 层”章节定义 API 接口对数据进行写入或读出操作。当 IO 管脚被复用为 7816\SPI\I2C 接口使用时，需配合对应章节接口使用。

2.2.5.1 术语定义

无。

2.2.5.2 数据结构定义

- IO HAL 层初始化结构体定义

```
typedef struct
{
    unsigned int    PinPos;
    unsigned int    Alternate;
    unsigned int    Pull;
    unsigned int    Driver;
    unsigned int    OpenDrain;
} IO_InitTypeDef;
```

2.2.5.3 函数接口描述

2.2.5.3.1 IO 配置 API

2.2.5.3.1.1 功能介绍

用户在使用此接口后，可以完成以下两个功能：

- 完成设置 IO 引脚复用功能，IO 引脚可复用为 7816\SPI\I2C\GPIO\SWD 接口；
- 完成 IO 引脚属性配置，包含上/下拉、开漏/推挽模式等；

2.2.5.3.1.2 接口定义

函数接口	void HAL_IO_Init(IO_InitTypeDef *Init)
输入	➢ Init：指向 IO 初始化结构 IO_InitTypeDef
输出	无
返回值	无
说明	➢ 用户在调用 API 前，需要先对结构 IO_InitTypeDef 进行初始化。

仅用于东信开发使用

2.2.5.3.2 IO 复位 API

2.2.5.3.2.1 功能介绍

复位 GPIO 接口，即恢复 IO 寄存器值到芯片复位值。

2.2.5.3.2.2 接口定义

函数接口	void HAL_IO_DeInit(unsigned int PinPos)
输入	➢ PinPos：对应的 GPIO 引脚号（本项目，PinPos 取值 0~7）；
输出	无
返回值	无
说明	无

2.2.6 其他类 API

2.2.6.1 概述

本类 API 包括：获取工厂码函数、读取芯片唯一序列号函数、时钟配置函数、设置 standby 模式等。

2.2.6.2 函数接口描述

2.2.6.2.1 获取工厂码 API

2.2.6.2.1.1 功能介绍

获取工厂码。

2.2.6.2.1.2 接口定义

函数接口	void HED_GetFactoryCode(unsigned char *FactoryCode)
输入	➢ FactoryCode：输出的芯片工厂码数据在内存 RAM 中起始地址
输出	➢ FactoryCode 地址中的内容
返回值	无
说明	➢ 读出数据长度为 40 字节。

2.2.6.2.2 读取芯片唯一序列号

2.2.6.2.2.1 功能介绍

获取芯片唯一序列号。

2.2.6.2.2.2 接口定义

函数接口	unsigned int HED_GetChipId(unsigned char *ChipId, unsigned int DataLen)
输入	➢ ChipId：输出的芯片唯一序列号数据在内存 RAM 中起始地址； ➢ DataLen：输出的芯片唯一序列号长度，7≤DataLen≤8。

仅用于东信开发使用

输出	➢ 7~8 字节芯片唯一序列号。
返回值	➢ 0x00000000：读取成功； ➢ 0x00000001：工厂码数据错误； ➢ 0xFFFFFFFF：输入长度非法。
说明	➢ 读取唯一序列号长度用户可指定，但是必须 7≤DataLen≤8； ➢ 该函数的执行时 CPU 必须处于特权模式（Privileged）； ➢ 该函数的执行不影响用户 COS 的运行环境配置。

2.2.6.2.3 读取芯片唯一序列号库版本

2.2.6.2.3.1 功能介绍

供 COS 获取芯片唯一序列号代码库版本。

2.2.6.2.3.2 接口定义

函数接口	unsigned int HED_ChipIDAPI_Version(void)
输入	无
输出	无
返回值	➢ 4 字节的芯片唯一序列号代码库版本
说明	➢ 版本格式定义：如读取版本号为 0xABCDMNXY： ● 0xAB：RFU（固定为 0x00）； ● 0xCD：表示芯片 Foundry 信息； ● 0xMN：表示不同工艺下，芯片唯一序列号生成规则的版本[注 1]； ● 0xXY：表示不同工艺下，芯片唯一序列号代码库版本[注 2]。

注 1：芯片唯一序列号生成规则的具体版本，请参见文档《038A-2018 芯片序列号生成规则（正式发布版）》；

注 2：芯片唯一序列号代码库版本，起始版本为 V1.0，相同芯片唯一序列号生成规则下，每次代码升级版本号递增；如芯片唯一序列号生成规则的版本升级，芯片唯一序列号代码库版本恢复至起始版本 V1.0。

2.2.6.2.4 CRC 算法 API

2.2.6.2.4.1 功能介绍

计算 RAM/FLASH 中指定数据的校验和。

模块可实现 16/32 位 CRC（Cyclic Redundancy Code）计算功能，分别使用生成多项式：

x16+x12+x5+1（CRC16-CCITT，0x1021）

x32+x26+x23+x22+x16+x12+x11+x10+x8+x7+x5+x4+x2+x+1（CRC32，0x04c1_1db7），

仅用于发行开发使用

该多项式基于 ISO/IEC 13239 标准

不同参数配置下的典型协议类型：

参数配置	典型的协议类型											
	CRC32		CRC-CCITT		TypeA		TypeB		JavaTCK		SWP	
输入数据类别	I 类	II 类	I 类	II 类	I 类	II 类	I 类	II 类	I 类	II 类	I 类	II 类
SWAPOUT：输出数据大小端转换	否	否	否	否	是	是	是	否	是	否	否	否
SWAPIN：输入数据大小端转换	是	否	是	是	是	是	是	是	是	否	是	是
REFOUT：输出数据位序颠倒	否	否	是	是	否	否	否	否	是	否	是	否
XOROUT：输出数据按位取反	是	否	否	否	否	否	是	否	是	否	否	否
REFIN：输入数据字节内位序颠倒	否	否	是	是	否	否	否	否	是	是	是	是
XORIN：输入数据按位取反	否	是	否	否	否	否	否	是	否	是	否	否
CRCPOLY：多项式	0	0	1	1	1	1	1	1	1	1	1	1
初值	0xffffffff		0xffff		0x6363		0xffff					
CRC32CTRL 配置值	**0x28**	**0x02**	**0x35**	**0x35**	**0x61**	**0x61**	**0x69**	**0x23**	**0x7d**	**0x07**	**0x35**	**0x25**

仅用于东信开发使用

软件对以上典型协议类型做如下宏定义：

#define CRC_TYPE_CRC32 (CRC_INPUT_ENDIAN_CONVER | CRC_OUTPUTDATA_BIT_INVERSION | CRC_POLY_CRC32)

#define CRC_TYPE_CCITT (CRC_INPUT_ENDIAN_CONVER | CRC_OUTPUTDATA_BIT_REVERSAL | CRC_INPUTDATA_BYTE_BITREVERSAL | CRC_POLY_CRC16_CCITT)

#define CRC_TYPE_TYPE_A (CRC_OUTPUT_ENDIAN_CONVER | CRC_INPUT_ENDIAN_CONVER | CRC_POLY_CRC16_CCITT)

#define CRC_TYPE_TYPE_B (CRC_OUTPUT_ENDIAN_CONVER |

CRC_INPUT_ENDIAN_CONVER | CRC_OUTPUTDATA_BIT_INVERSION | CRC_POLY_CRC16_CCITT)

#define CRC_TYPE_JAVATCK (CRC_OUTPUT_ENDIAN_CONVER | CRC_INPUT_ENDIAN_CONVER | CRC_OUTPUTDATA_BIT_REVERSAL | CRC_OUTPUTDATA_BIT_INVERSION | CRC_INPUTDATA_BYTE_BITREVERSAL | CRC_POLY_CRC16_CCITT)

#define CRC_TYPE_SWP (CRC_INPUT_ENDIAN_CONVER | CRC_OUTPUTDATA_BIT_REVERSAL | CRC_INPUTDATA_BYTE_BITREVERSAL | CRC_POLY_CRC16_CCITT)

2.2.6.2.4.2 接口定义

CRC16：

函数接口	unsigned short HED_CRC16(unsigned short IV, unsigned int DataLen, unsigned char *DataBuf, unsigned char type)
输入	➢ IV：输入的初始值 IV； ➢ DataLen：待进行 CRC 校验的数据长度，如果该参数为 0，则该函数仅返回输入的 IV 值，不做任何运算； ➢ DataBuf：输入数据缓冲； ➢ type： ● CRC_TYPE_CCITT ● CRC_TYPE_TYPE_A ● CRC_TYPE_TYPE_B ● CRC_TYPE_JAVATCK ● CRC_TYPE_SWP
输出	无
返回值	2 字节校验码（大端格式）。
资源使用	----
说明	----

CRC32:

函数接口	unsigned int HED_CRC32(unsigned int IV, unsigned int DataLen, unsigned char *DataBuf, unsigned char type)
输入	➢ IV：输入的初始值 IV； ➢ DataLen：待进行 CRC 校验的数据长度，如果该参数为 0，则该函数

仅用于华信开发使用

	仅返回输入的 IV 值，不做任何运算； ➢ DataBuf：输入数据缓冲； ➢ type： ● CRC_TYPE_CRC32
输出	无
返回值	4 字节校验码（大端格式）。
资源使用	----
说明	----

函数接口	unsigned int HED_CRC(unsigned int IV, unsigned int DataLen, unsigned char *DataBuf, unsigned char type)
输入	➢ IV：输入的初始值 IV； ➢ DataLen：待进行 CRC 校验的数据长度，如果该参数为 0，则该函数仅返回输入的 IV 值，不做任何运算； ➢ DataBuf：输入数据缓冲； ➢ type： ● bit7: RFU ● bit6（OUTPUT_REV）：输出数据大小端转换，如果此位置“1”，在读取 16 位校验结果时，校验结果位顺序高低颠倒输出 ● bit5: 输入数据大小端转换 ● bit4: 输出数据位序高低颠倒控制 ● bit3: 输出结果按位取反 ● bit2（INPUT_REV）：输入数据字节内位序高低颠倒控制，如果此位置“1”，计算 CRC 时，将输入数据各字节的位顺序高低颠倒后用于计算 ● bit1: 输入数据按位取反 ● bit0: 1，16 位 CRC 计算，0，32 位 CRC 计算
输出	无
返回值	2 或 4 字节校验码（大端格式）。
资源使用	----
说明	➢ 此接口可实现 16/32 位 CRC 计算功能：

仅用于华信开发使用

2.2.6.2.5 模块时钟管理 API

2.2.6.2.5.1 功能介绍

配置 AHB1_CLK/AHB2_CLK/APB_CLK 总线下模块时钟开启和关闭

2.2.6.2.5.2 接口定义

函数接口	HAL_StatusTypeDef HAL_CKMU_AXBControl(BUS_CLKTypeDef bus, uint32_t clockpos, uint32_t state)
输入	➢ bus：总线类型，AHB1_CLK/AHB2_CLK/APB_CLK； ➢ clockpos：要控制的模块时钟位,与 SPEC 寄存器位对应； ➢ state：0 关闭模块时钟，其它值开启模块时钟；
输出	无
返回值	➢ HAL_StatusTypeDef 定义值
资源使用	----
说明	➢ 模块内对 AHB1_CLK/AHB2_CLK/APB_CLK 进行打开/关闭

2.2.6.2.6 读取固件 API 整体版本

2.2.6.2.6.1 功能介绍

供 COS 获取 4 字节固件 API 整体版本，主要包括获取掩膜算法 API 版本，获取掩膜 NVM API 版本，获取随机数库 API 版本，获取芯片唯一序列号库 API 版本，获取增强稳定性方案库 API 版本。

2.2.6.2.6.2 接口定义

函数接口	unsigned int HED_FWAPI_TotalVersion(unsigned int APIIndexFlag)
输入	➢ APIIndexFlag：选择获取 4 字节不同 API 类型版本号： ● 0x00：代表获取掩膜算法 API 版本； ● 0x01：代表获取掩膜 NVM API 版本； ● 0x03：代表获取随机数库 API 版本； ● 0x04：代表获取唯一序列号库 API 版本； ● 0x05：代表获取增强稳定性方案库 API 版本； ● 0xFFFFFFFF：代表获取上述所有 API 类型（即 0x0~0x05 对应的 API 类型）版本号的 CRC 值：其中高两字节填充 0x0000，低两字节为 CRC 校验值（2 字节校验码（大端格式））； ● 其余值：获取增强稳定性方案库 API 版本。
输出	无
返回值	4 字节的 API 版本

仅用于芯信开发使用

资源使用	----
说明	➢ 当 APIIndexFlag 为 FFFFFFFF，即 CRC 计算时，符合标准 TYPEB 协议 CRC-16（初始值为 FFFFH）。

2.3 增强芯片稳定性方案 API

2.3.1 概述

为了提高 FLASH 中数据的可靠性，**防止程序在外界干扰（特别是上下电期间容易被干扰）时 PC 指针跑飞**等情况下，FLASH 中数据被意外改写，本节中的 API 配合 NVM 类 API，提高芯片稳定性。

2.3.2 函数接口描述

2.3.2.1 软件开关初始化函数

2.3.2.1.1 功能介绍

关闭 NVM 操作开关：清零所有 NVM 操作开关，禁止 NVM 操作。

2.3.2.1.2 接口定义

函数接口	void MACRO_Init_St(void)
输入	无
输出	无
返回值	无
说明	➢ 本函数的具体用法请参考《增强芯片稳定性方案》。

2.3.2.2 关闭软件开关 12 函数

2.3.2.2.1 功能介绍

关闭 NVM 操作开关 1、2。

2.3.2.2.2 接口定义

函数接口	void closeWrSwitch12(void)
输入	无
输出	无
返回值	无
说明	➢ 本函数的具体用法请参考《增强芯片稳定性方案》

仅用于东信开发使用

2.3.2.3 关闭软件开关 34 函数

2.3.2.3.1 功能介绍

关闭 NVM 操作开关 3、4。

2.3.2.3.2 接口定义

函数接口	void closeWrSwitch34(void)
输入	无
输出	无
返回值	无
说明	➢ 本函数的具体用法请参考《增强芯片稳定性方案》。

2.3.2.4 关闭软件开关 234 函数

2.3.2.4.1 功能介绍

关闭 NVM 操作开关 2、3、4。

2.3.2.4.2 接口定义

函数接口	void closeWrSwitch234(void)
输入	无
输出	无
返回值	无
说明	➢ **如果 COS 不支持进入 Standby，需要调用本函数关闭 NVM 操作开关，否则，不需要单独调用本函数，直接调用增强稳定性** setStandby_st **函数即可完成关闭 NVM 操作开关和进入 Standby。**

2.3.2.5 打开软件开关 1 函数

2.3.2.5.1 功能介绍

开启 NVM 操作开关 1。

2.3.2.5.2 接口定义

函数接口	void openWrSwitch1(void)
输入	无
输出	无
返回值	无
说明	◆ 本函数的具体用法请参考《增强芯片稳定性方案》。

仅用于东信开发使用

2.3.2.6 打开软件开关 2 函数

2.3.2.6.1 功能介绍

开启 NVM 操作开关 2。

2.3.2.6.2 接口定义

函数接口	void openWrSwitch2(void)
输入	无
输出	无
返回值	无
说明	➢ 本函数的具体用法请参考《增强芯片稳定性方案》。

2.3.2.7 打开软件开关 3 函数

2.3.2.7.1 功能介绍

开启 NVM 操作开关 3、4。

2.3.2.7.2 接口定义

函数接口	void openWrSwitch3(void)
输入	无
输出	无
返回值	无
说明	➢ 本函数的具体用法请参考《增强芯片稳定性方案》。

2.3.2.8 读取增强稳定性方案库版本

2.3.2.8.1 功能介绍

供 COS 获取增强稳定性方案库版本。

2.3.2.8.2 接口定义

函数接口	unsigned int HED_StSwitchAPI_Version(void)
输入	无
输出	无
返回值	➢ 4 字节的增强稳定性方案库版本
说明	➢ 版本格式定义，如读取版本号为 0xABCDMNXY： ● 0xABCDMN：RFU（固定为 0x000000）； ● 0xXY：表示稳定性方案库版本。

仅用于东信开发使用

2.4 随机数类 API

2.4.1 概述

随机数类 API 包括：获取真随机数函数、获取伪随机数函数、 随机数卡方检测、随机数扑克检测。

2.4.2 函数接口描述

2.4.2.1 随机数卡方检测

2.4.2.1.1 函数说明

用于测试随机数检测。

2.4.2.1.2 接口定义

函数接口	unsigned int HED_RNG1Check(void)
输入	无
输出	无
返回值	➢ 0xCA793B4A：随机数检测通过； ➢ 0xB7359A3B：随机数检测失败； ➢ 其他值等同于“随机数检测失败”。
说明	➢ 对真随机数进行检测； ➢ API 内部采用 χ^2 检验方法，通过硬件计数和统计的方式实现； ➢ API 内部开启 χ^2 检验的硬件计数和统计功能； ➢ API 内不负责打开 RNG1 模块门控时钟，芯片默认开启； ➢ 若检测失败，建议用户再进行一次检测。如果第二次检测成功，则按照正常流程处理，如果失败，则返回错误或抛出异常。

2.4.2.2 获取真随机数 API

2.4.2.2.1 功能介绍

获取真随机数。

2.4.2.2.2 接口定义

函数接口	unsigned int HED_GetRandom_Enhanced(unsigned char *RandomData, unsigned int DataLen, unsigned int *paraChkSum)
输入	➢ RandomData：输出的随机数数据在内存 RAM 中的起始地址； ➢ DataLen：输出的随机数数据长度； ➢ paraChkSum：RAM 中的校验参数地址指针（校验参数占用空间为一

仅用于客户开发使用

	个字）。
输出	➢ RandomData：将实际输出的随机数写入该指针指向的地址 ➢ paraChkSum：指向校验参数，函数正常状返回时，校验参数等于输出结果地址+数据长度。如果函数返回值不是“执行成功”，则此参数无意义。若校验参数不正确，则说明函数在执行过程中有可能收到攻击，输出结果不可信。
返回值	➢ 0xBC2679B3：随机数生成成功； ➢ 0x927C45A5：随机数生成失败（卡方自检失败或芯片受到干扰或故障攻击）； ➢ 其他值等同于“随机数生成失败”。
说明	➢ API 执行前后，保持 RNG1 的门控时钟状态不变。
安全使用说明	➢ 仅当“返回值是成功”+“paraChkSum 合法”时，本函数的输出才可信，否则说明 API 执行过程中可能受到安全攻击。

2.4.2.3 获取伪随机数 API

2.4.2.3.1 功能介绍

获取伪随机数。

2.4.2.3.2 接口定义

函数接口	unsigned int HED_GetRandom(unsigned char *RandomData,unsigned int DataLen, unsigned int *paraChkSum)
输入	➢ RandomData：输出的随机数数据在内存 RAM 中的起始地址； ➢ DataLen：输出的随机数数据长度； ➢ paraChkSum：RAM 中的校验参数地址指针（校验参数占用空间为一个字）。
输出	➢ RandomData：将实际输出的随机数写入该指针指向的地址； ➢ paraChkSum：指向校验参数，函数正常状返回时，校验参数等于输出结果地址+数据长度。如果函数返回值不是“执行成功”，则此参数无意义。若校验参数不正确，则说明函数在执行过程中有可能收到攻击，输出结果不可信。
返回值	➢ 0xBC2679B3：随机数生成成功； ➢ 0x927C45A5：随机数生成失败（卡方自检失败或芯片受到干扰或故障攻击）； ➢ 其他值等同于“随机数生成失败”。

仅用于华信开发使用

说明	➢ API 执行前后，保持 RNG1 的门控时钟状态不变； ➢ API 内使用 DES 算法产生伪随机数，执行 API 后 DES 模块门控时钟恢复为 API 执行前的配置。
安全使用说明	➢ 仅当“返回值是成功”+“paraChkSum 合法”时，本函数的输出才可信，否则说明 API 执行过程中可能受到安全攻击。

2.4.2.4 读取随机数库版本 API

2.4.2.4.1 功能介绍

供 COS 获取随机数库版本。

2.4.2.4.2 接口定义

函数接口	unsigned int HED_RNGAPI_Version(void)
输入	无
输出	无
返回值	➢ 版本
说明	无

2.5 对称算法 GCM 模式 API

2.5.1 功能介绍

本节函数可处理多字节的 GCM 模式的对称算法加解密。AES 算法的 GCM 模式通过外部函数指针指定。

2.5.2 数据结构

```
typedef struct GCM_Context_Key
{
    uint32_t Nr;
    uint8_t rk[240];
    void (* pKeyExpansionFunc)(uint8_t *, uint8_t *, uint16_t);
}GCM_Context_Key;

typedef struct GCM_Context
{
    uint64_t HL[16];                    /*!< Precalculated HTable low. */
    uint64_t HH[16];                    /*!< Precalculated HTable high. */
```

仅用于东信开发使用

```
    uint64_t len;                   /*!< The total length of the encrypted data. */
    uint64_t add_len;               /*!< The total length of the additional data. */
    unsigned char base_ectr[16];    /*!< The first ECTR for tag. */
    unsigned char y[16];            /*!< The Y working value. */
    unsigned char buf[16];          /*!< The buf working value. */
    int mode;                       /*!< The operation to perform:
                                         #GCM_ENCRYPT or
                                         #GCM_DECRYPT. */
    // starts fun para
    int32_t mode_;
    const uint8_t *iv_;
    size_t iv_len_;
    // updata ad fun para
    const uint8_t *ad_;
    size_t ad_len_;
    // updata fun para
    const uint8_t *input_;
    size_t input_length_;
    uint8_t *output_;
    size_t output_size_;
    size_t *output_length_;
    // finish fun para
    uint8_t *tag_;
    size_t tag_len_;
    void (* pAlgEcbFunc)(uint8_t *, uint8_t *, uint8_t);
}GCM_Context;
```

2.5.3 函数接口描述

2.5.3.1 初始化结构体

2.5.3.1.1 功能介绍

初始化内部使用的结构体。调用此函数前，需要用户自行初始化 ctx 成员中的 pAlgEcbFunc 函数指针。

仅用于东信开发使用

2.5.3.1.2 接口定义

函数接口	void HED_GCMInit(GCM_Context *ctx);
输入	ctx：内部使用的结构体
输出	无
返回值	无
资源使用	无
说明	先调用此函数再调用其他的

2.5.3.2 密钥初始化函数

2.5.3.2.1 功能介绍

初始化 key 结构体。调用此函数前，需要用户自行初始化 ctx_key 成员中的 pKeyExpansionFunc 函数指针。Keybits 根据实际需求传入，如 128、192 或 256。对于只有 128bit 密钥长度的算法，将忽视此值的传入，固定执行 128bit 的密钥扩展。

2.5.3.2.2 接口定义

函数接口	uint32_t HED_GCMSetkey(GCM_Context *ctx, GCM_Context_Key *ctx_key, const unsigned char *key, unsigned int keybits);
输入	ctx：内部使用的结构体。 ctx_key：内部使用的结构体 key：用户输入的原始密钥 keybits：密钥的 bit 长度
输出	无
返回值	0 代表成功，非 0 代表错误
资源使用	无
说明	

2.5.3.3 起始函数

函数接口	uint32_t HED_GCMStarts(GCM_Context *ctx,

仅用于东信开发使用

	GCM_Context_Key *ctx_key);
输入	ctx：内部使用的结构体。用户调用此函数时需要给结构体成员中的以下成员赋初值。 mode_：模式，表明加密还是解密 iv_：IV 的地址。Iv 即规范中的 nonce iv_len_：IV 的长度，取值范围是(0,2^61)。 ctx_key：密钥相关结构体。
输出	无
返回值	0 代表成功，非 0 代表错误
资源使用	无
说明	先调用此函数再调用其他的

2.5.3.4 更新 ADD 函数

函数接口	uint32_t HED_GCMUpdataAd(GCM_Context *ctx);
输入	ctx：内部使用的结构体。用户调用此函数时需要给结构体成员中的以下成员赋初值。 ad_：单次的 add 的起始地址 ad_len_：单次的 add 的长度。Add 支持多次传入，取值范围是[0,2^61]。
输出	无
返回值	0 代表成功，非 0 代表错误
资源使用	无
说明	先调用此函数再调用其他的

2.5.3.5 更新密文函数

函数接口	uint32_t HED_GCMUpdata(GCM_Context *ctx, GCM_Context_Key *ctx_key);
输入	ctx：内部使用的结构体。用户调用此函数时需要给结构体成员中的以下成员赋初值。 input_：明文的地址 input_len_：明文的长度 output_：输出的地址 output_size_：输出的长度。

仅用于华信开发使用

	output_len_：如果函数执行成功，该指针指向的值等于 input_len_，否则该指针指向的值非预期。 ctx_key：密钥相关结构体。
输出	无
返回值	0 代表成功，非 0 代表错误
资源使用	无
说明	先调用此函数再调用其他的

2.5.3.6 完成函数

函数接口	uint32_t HED_GCMFinish(GCM_Context *ctx);
输入	ctx：内部使用的结构体。用户调用此函数时需要给结构体成员中的以下成员赋初值。 tag_：tag 的地址 tag_len_：tag 的长度，取值范围是[4,16]。 ctx_key：密钥相关结构体。
输出	无
返回值	0 代表成功，非 0 代表错误
资源使用	无
说明	先调用此函数再调用其他的

2.5.3.7 获取版本号函数

函数接口	unsigned int HED_GCMAPI_Version(void);
输入	无
输出	无
返回值	4 字节版本号
资源使用	无
说明	无

仅用于华信开发使用

2.6 对称算法 CCM 模式 API

2.6.1 功能介绍

本节函数可处理多字节的 CCM 模式的对称算法加解密。AES 算法的 CCM 模式通过外部函数指针指定。在《CCM 模式算法 API 手册》中不限制算法类型。

2.6.2 数据结构

```
typedef struct CCM_Context_Key
{
    uint32_t Nr;
    uint8_t rk[240];
    void (* pKeyExpansionFunc)(uint8_t *, uint8_t *, uint16_t);
}CCM_Context_Key;

typedef struct CCM_Context
{
    uint8_t y[16];      /*!< The Y working buffer */
    uint8_t ctr[16];    /*!< The counter buffer */
    //HED_cipher_context_t HED_PRIVATE(cipher_ctx);      /*!< The cipher context used. */
    size_t plaintext_len;     /*!< Total plaintext length */
    size_t add_len;           /*!< Total authentication data length */
    size_t tag_len;           /*!< Total tag length */
    size_t processed;         /*!< Track how many bytes of input data
                                   were processed (chunked input).
                                   Used independently for both auth data
                                   and plaintext/ciphertext.
                                   This variable is set to zero after
                                   auth data input is finished. */
    uint8_t q;                /*!< The Q working value */
    uint8_t mode;             /*!< The operation to perform:
                                   #HED_CCM_ENCRYPT or
                                   #HED_CCM_DECRYPT */
    int32_t state;            /*!< Working value holding context's
                                   state. Used for chunked data
                                   input */
    // starts fun para
    int32_t mode_;
    const uint8_t *iv_;
    size_t iv_len_;
```

仅用于华大电子开发使用

```
    // set lengths fun para
    size_t total_ad_len_;
    size_t plaintext_len_;
    size_t tag_len_;
    // updata ad fun para
    const uint8_t *ad_;
    size_t ad_len_;
    // updata fun para
    const uint8_t *input_;
    size_t input_len_;
    uint8_t *output_;
    size_t output_size_;
    size_t *output_len_;
    // finish fun para
    uint8_t *tag_;
    void (* pAlgEcbFunc)(uint8_t *, uint8_t *, uint8_t);
}CCM_Context;
```

2.6.3 函数接口描述

2.6.3.1 初始化结构体

2.6.3.1.1 功能介绍

初始化内部使用的结构体。调用此函数前，需要用户自行初始化 ctx 成员中的 pAlgEcbFunc 函数指针。

2.6.3.1.2 接口定义

函数接口	void HED_CCMInit(CCM_Context *ctx);
输入	ctx：内部使用的结构体
输出	无
返回值	无
资源使用	无
说明	先调用此函数再调用其他的

仅用于东信开发使用

2.6.3.2 密钥初始化函数

2.6.3.2.1 功能介绍

初始化 key 结构体。调用此函数前，需要用户自行初始化 ctx_key 成员中的 pKeyExpansionFunc 函数指针。Keybits 根据实际需求传入，如 128、192 或 256。对于只有 128bit 密钥长度的算法，将忽视此值的传入，固定执行 128bit 的密钥扩展。

2.6.3.2.2 接口定义

函数接口	uint32_t HED_CCMSetkey(CCM_Context_Key *ctx_key, const uint8_t *key, uint32_t keybits)
输入	ctx_key：内部使用的结构体 key：用户输入的原始密钥 keybits：密钥的 bit 长度
输出	无
返回值	0 代表成功，非 0 代表错误
资源使用	无
说明	

2.6.3.3 起始函数

函数接口	uint32_t HED_CCMStarts(CCM_Context *ctx, CCM_Context_Key *ctx_key)
输入	ctx：内部使用的结构体。用户调用此函数时需要给结构体成员中的以下成员赋初值。 mode_：模式，表明加密还是解密 iv_：IV 的地址。Iv 即规范中的 nonce iv_len_：IV 的长度，取值范围是[8,12]。 ctx_key：密钥相关结构体。
输出	无
返回值	0 代表成功，非 0 代表错误
资源使用	无
说明	先调用此函数再调用其他的

仅用于东信开发使用

2.6.3.4 设置长度函数

函数接口	uint32_t HED_CCMSetLengths(CCM_Context *ctx, CCM_Context_Key *ctx_key)
输入	ctx：内部使用的结构体。用户调用此函数时需要给结构体成员中的以下成员赋初值。 total_ad_len_：add 的总长度，取值范围是(0,0xFF00)。 plaintext_len_：明文的总长度 tag_len_：tag 的总长度，取值集合是{4,6,8,10,12,14,16} ctx_key：密钥相关结构体。
输出	无
返回值	0 代表成功，非 0 代表错误
资源使用	无
说明	先调用此函数再调用其他的

2.6.3.5 更新 ADD 函数

函数接口	uint32_t HED_CCMUpdataAd(CCM_Context *ctx, CCM_Context_Key *ctx_key)
输入	ctx：内部使用的结构体。用户调用此函数时需要给结构体成员中的以下成员赋初值。 ad_：单次的 add 的起始地址 ad_len_：单次的 add 的长度。Add 支持多次传入，只要总长度等于之前传入的 total_ad_len_即可。 ctx_key：密钥相关结构体。
输出	无
返回值	0 代表成功，非 0 代表错误
资源使用	无
说明	先调用此函数再调用其他的

2.6.3.6 更新密文函数

函数接口	uint32_t HED_CCMUpdata(CCM_Context *ctx, CCM_Context_Key *ctx_key)
输入	ctx：内部使用的结构体。用户调用此函数时需要给结构体成员中的以

仅用于华信开发使用

	下成员赋初值。 input_：明文的地址 input_len_：明文的长度 output_：输出的地址 output_size_：输出的长度 output_len_：如果函数执行成功，该指针指向的值等于 input_len_，否则该指针指向的值非预期。 ctx_key：密钥相关结构体。
输出	无
返回值	0 代表成功，非 0 代表错误
资源使用	无
说明	先调用此函数再调用其他的

2.6.3.7 完成函数

函数接口	uint32_t HED_CCMFinish(CCM_Context *ctx, CCM_Context_Key *ctx_key)
输入	ctx：内部使用的结构体。用户调用此函数时需要给结构体成员中的以下成员赋初值。 tag_：tag 的地址 ctx_key：密钥相关结构体。
输出	无
返回值	0 代表成功，非 0 代表错误
资源使用	无
说明	先调用此函数再调用其他的

2.6.3.8 获取版本号函数

函数接口	unsigned int HED_CCMAPI_Version(void);
输入	无
输出	无
返回值	4 字节版本号
资源使用	无
说明	无

仅用于华大信安开发使用

3 芯片掩膜算法 API

3.1 算法类 API

3.1.1 概述

算法类 API 包括： AES、DES/TDES、ECC、SHA、CRC、X25519。

3.1.2 函数接口描述

3.1.2.1 对称算法 mask 设置 API

3.1.2.1.1 功能介绍

本函数可以设置对称算法的 MASK 掩码，本芯片支持 DES，AES 两个算法。

3.1.2.1.2 接口定义

函数接口	typedef enum { DESKEYMsk = 4, TDES2KEYMsk = 5, TDES3KEYMsk = 6, AES128KEYMsk = 7, AES192KEYMsk = 8, AES256KEYMsk = 9, } SymmetricKeyMask; unsigned int HED_SymmetricKeyMask(unsigned int *iKeyMaskPtr, SymmetricKeyMask KeyMask)
输入	➢ iKeyMaskPtr：密钥运算的 MASK 值，根据算法不同，输入的长度也不同，但是和对应算法的实际运算的秘钥长度相同，如 DES 运算时，这里的 MASK 需要输入 8 字节，AES256bit 密钥运算时，这里的 MASK 需要输入 32 字节。真正参与运算的密钥为用户调用对应算法 Block 函数时输入的密钥与这个 MASK 的异或值 ➢ KeyMask：枚举值，代表用户调用此函数想设置哪个算法的密钥 MASK。
输出	无
返回值	➢ 0xB7C8D9EA，执行成功； ➢ 0x932057CE，输入数据错误（输入枚举值错误）； ➢ 其他值等同于“输入数据错误”。
资源使用	----
说明	➢ 必须在调用算法运算函数之前调用此函数； ➢ 输入数据的存放顺序为：高字节存放低地址； ➢ 本算法执行前后，保持本算法 IP 的门控时钟状态不变。 ➢ 如果用户调用此函数传入了非 0 值的 mask 之后进行某个算法的任意模式的运算，当再次运算时不希望使用 mask 时，无论再次运算时是否切换算

仅用于芯信开发使用

	法，都需要调用此函数，并传入全 0 值的 mask 进行内部 mask 值清空。否则硬件会继续使用之前的 mask 值对秘钥进行脱掩。

3.1.2.2 DES/TDES 算法 API

3.1.2.2.1 功能介绍

本节函数可处理多字节的 ECB 或 CBC 模式的 DES/TDES 加解密。

3.1.2.2.2 接口定义

函数接口	typedef struct { unsigned int *iData Ptr; unsigned int DataLen; unsigned int *iKeyPtr; unsigned int *IVPtr; unsigned int *oDataPtr; unsigned char Type; usingned int paraChkSum; } DES_Parameters, TDES_Parameters; unsigned int HED_DESBlock(DES_Parameters *DES_Param) unsigned int HED_TDESBlock_2Key(TDES_Parameters *TDES_Param)
输入	DES_Param：DES/3DES 运算结构数据指针： ➢ iDataPtr：RAM 中的待加密或解密的数据指针； ➢ DataLen：数据的长度（应为 8 的倍数）； ➢ iKeyPtr：8 字节密钥(DES)/16 字节密钥（TDES）(TDES 密钥根据 KEY 类型的不同，2KEY 为 8 字节 KEY1+8 字节的 KEY2)； ➢ IVPtr：RAM 中的 8 字节初始向量指针（CBC 模式下有效）； ➢ oDataPtr：RAM 中的加密或解密的结果数据指针； ➢ Type：运算类型选择（Type 的具体定义请参阅附录 A）： ● 0x00，ECB 模式加密； ● 0x40，ECB 模式解密； ● 0x20，CBC 模式加密； ● 0x60，CBC 模式解密。 注 1：对于非本 API 支持的 Type 类型，API 不保证输出结果的正确性； 注 2：本 API 无内外部参数的情况。
输出	➢ oDataPtr 指向的内容； ➢ paraChkSum：校验参数，函数正常返回时，校验参数等于输入消息地址+输出结果地址+数据长度+密钥地址+Type 值。如果函数返回值不是“执行成功”，则此参数无意义。若校验参数不正确，则说明函数在执行过程中有可能收到攻击，输出结果不可信。
返回值	➢ 0xB7C8D9EA，执行成功； ➢ 0x932057CE，输入数据错误（输入数据长度为 0 或非 8 字节整数倍）；

仅用于华大电子开发使用

	➢ 0x48372615，运算错误（芯片受到干扰或故障攻击导致运算结果不正确）； ➢ 其他值等同于“运算错误”。
资源使用	-----
说明	➢ 对于 CBC 模式，IVPtr 为输入初始化数据；对于 ECB 模式，IVPtr 无用； ➢ 若为 CBC 模式，oDataPtr 也把过程中所有数据输出； ➢ 输入和输出数据的存放顺序为：高字节存放低地址； ➢ 输出可以和输入指针地址相同； ➢ 算法 API 执行前后，保持本算法 IP 的门控时钟状态不变。
安全使用说明	➢ 仅当“返回值是成功”+“paraChkSum 合法”时，本函数的输出才可信，否则说明 API 执行过程中可能受到安全攻击。

3.1.2.3 AES 算法 API

3.1.2.3.1 功能介绍

本函数可处理多字节的 ECB、CBC、CFB、OFB、CTR 模式的 AES 加解密和 MAC 运算。

3.1.2.3.2 接口定义

函数接口	typedef struct { unsigned int *iDataPtr; unsigned int DataLen; unsigned int *iKeyPtr; unsigned int *IVPtr; unsigned int *oDataPtr; unsigned char Type; usingned int paraChkSum; } AES_Parameters; unsigned int HED_AESBlock(AES_Parameters *AES_Param)
输入	AES _Param：AES 运算结构数据指针： ➢ iDataPtr：RAM 中的待加密或解密的数据指针； ➢ DataLen：数据的长度（应为 16 的倍数）； ➢ iKeyPtr：16 字节或 24 字节或 32 字节的密钥指针； ➢ IVPtr：RAM 中的 16 字节初始向量指针（CBC/OFB/CFB/MAC 模式下有效）；CTR 模式下为 16 字节的初始值和 4 个字节的增量值； ➢ oDataPtr：RAM 中的加密或解密的结果数据指针； ➢ Type：运算类型选择（Type 的具体定义请参阅附录 A）： ● 0x00，ECB 模式加密； ● 0x40，ECB 模式解密； ● 0x20，CBC 模式加密； ● 0x60，CBC 模式解密； ● 0x10，CFB 模式加密；

仅用于东信开发使用

	● 0x50，CFB 模式解密； ● 0x30，OFB 模式加密； ● 0x70，OFB 模式解密； ● 0x08，CTR 模式加密； ● 0x48，CTR 模式解密； 其中 Type 参数的 bit1bit0 用于表征 AES 的 key 类型： ● 00：128key ● 01：192key ● 10：256key 注 1：对于非本 API 支持的 Type 类型，API 不保证输出结果的正确性； 注 2：本 API 无内外部参数的情况。
输出	➢ oDataPtr 指向的内容； ➢ paraChkSum：校验参数，函数正常返回时，校验参数等于输入消息地址+输出结果地址+数据长度+密钥地址+Type 值。如果函数返回值不是“执行成功”，则此参数无意义。若校验参数不正确，则说明函数在执行过程中有可能收到攻击，输出结果不可信。
返回值	➢ 0xB7C8D9EA，执行成功； ➢ 0x932057CE，输入数据错误（输入数据长度为 0 或非 16 字节整数倍）； ➢ 0x48372615，运算错误（芯片受到干扰或故障攻击导致运算结果不正确）； ➢ 其他值等同于“运算错误”。
资源使用	----
说明	➢ 对于 CBC 模式，IVPtr 为输入初始化数据；对于 ECB 模式，IVPtr 无用； ➢ 若为 CBC 模式，oDataPtr 也把过程中所有数据输出； ➢ 输入和输出数据的存放顺序为：高字节存放低地址； ➢ 输出可以和输入指针地址相同； ➢ 算法 API 执行前后，保持本算法 IP 的门控时钟状态不变。
安全使用说明	➢ 仅当“返回值是成功”+“paraChkSum 合法”时，本函数的输出才可信，否则说明 API 执行过程中可能受到安全攻击。

仅供华信安使用

3.1.2.4 ECC 算法 API

3.1.2.4.1 概述

包含 ECC 密钥参数初始化、密钥生成、签名、验证、加密、解密、公私钥验证运算代码。

3.1.2.4.2 曲线参数说明

在本文中，椭圆曲线参数长度均以字长表示，记为 N。比如：

Fp-192：素数 p，系数 a、b，基点 G（x，y）其阶为 n。p，a，b，x，y，n 均为 24 字节，即 N=6。

Fp-224：素数 p，系数 a、b，基点 G（x，y）其阶为 n。p，a，b，x，y，n 均为 28 字节，即 N=7。

Fp-256：素数 p，系数 a、b，基点 G（x，y）其阶为 n。p，a，b，x，y，n 均为 32 字节，即 N=8。

Fp-384：素数 p，系数 a、b，基点 G（x，y）其阶为 n。p，a，b，x，y，n 均为 48 字节，即 N=12。

Fp-521：素数 p，系数 a、b，基点 G（x，y）其阶为 n。p，a，b，x，y，n 均为 66 字节，即 N=17。

本函数库支持常见的曲线参数，具体如下：

EC_brainpoolP192r1，EC_brainpoolP192t1，
EC_brainpoolP224r1，EC_brainpoolP224t1，
EC_brainpoolP256r1，EC_brainpoolP256t1，
EC_brainpoolP320r1，EC_brainpoolP320t1，
EC_brainpoolP384r1，EC_brainpoolP384t1，
EC_brainpoolP512r1，EC_brainpoolP512t1，
EC_nistp192，EC_nistp224，EC_nistp256，
EC_nistp384，EC_nistp521，
EC_SECG_PRIME_192K1，EC_SECG_PRIME_256K1，
EC_secp192r1，EC_secp224r1，EC_frp256V1

3.1.2.4.3 数据存储顺序

算法中用到的参数和数据中，均为大端方式，即低地址存放高字节。例如，对于 16 进制数据：0x11223344，应按照下面的顺序输入：pDataIn[0] = 0x11，pDataIn [1] = 0x22，pDataIn [2] = 0x33，pDataIn [3] = 0x44。

3.1.2.4.4 ECC 函数库接口参数

ECC 函数的参数初始化函数由结构体数据 ECC_Init_Parameters 传递。

ECC_Init_Parameters 的定义如下：

```
typedef struct
{
    const unsigned int      *iEccP;
    const unsigned int      *iEccA;
    const unsigned int      *iEccB;
    const unsigned int      *iEccR;
    const unsigned int      *iEccG;
    unsigned int            *oDataPtr;
    unsigned int            bitLen;
    unsigned int            hashType;
} ECC_Init_Parameters;
```

仅用于测试开发使用

ECC 函数的输入输出参数由结构体数据 ECC_Parameters 传递。ECC_Parameters 的定义如下：

```
typedef struct
{
    const unsigned char     *iDataPtr;
    unsigned char           *oDataPtr;
    const unsigned int      *rndPtr;
    unsigned int            *priKeyPtr;   //密钥生成会写入该区域，其它函数为只读方式使用
    unsigned int            *pubKeyPtr;   //密钥生成会写入该区域，其它函数为只读方式使用
    const unsigned int      *argPtr;
    unsigned int            paraChkSum;
} ECC_Parameters;
```

上述结构体定义中，priKeyPtr 和 pubKeyPtr 可以在头文件中修改为 const 类型，但是需要注意的是，在 ECC 密钥生成中要分配可写入 RAM 空间，否则会产生写数据异常。

表 3.1 成员变量说明

名称	类型	目标数据长度（字：32 位，N 为ECC 曲线参数的字长度）	描述
iDataPtr	unsigned char *	由输入确定	指向输入数据。
oDataPtr	unsigned char *	由输出确定	指向输出数据。
rndPtr	unsigned int *	N（若长度不足整字时，需要在高位补 0）	指向签名或解密中使用的随机数。
priKeyPtr	unsigned int *	N（若长度不足整字时，需要在高位补 0）	指向私钥，只在密钥生成函数中写入，其它函数中为只读方式使用。
pubKeyPtr	unsigned int *	N*2（若长度不足整字时，需要在高位补 0）	指向公钥，包含两部分（Px,Py），连续排列，只在密钥生成函数中写入，其它函数中位只读方式使用。
argPtr	unsigned int*	由参数确定	椭圆曲线参数选择，输入按格式组织的曲线参数，使用时只需要把曲线参数 lib 中对应曲线参数的首地址传入即可，也可以调用 HED_ECCInitPara 函数由基本参数生成。
paraChkSum	unsigned int	1	输出函数参数校验值

```
typedef struct
{
    unsigned int        byteLen_SharedInfo1;
    const void          *SharedInfo1;
    unsigned int        byteLen_SharedInfo2;
```

仅用于客户开发使用

```
    const void          *SharedInfo2;
} ECIES_Parameters;
```

表 3.2 ECIES_Parameters 成员变量说明

名称	类型	目标数据长度（字：32 位，N 为 ECC 曲线参数的字长度）	描述
byteLen_SharedInfo1	unsigned int	1	表示加解密双方共享数据 1 的长度，单位是字节。
SharedInfo1	const void *	由输入确定	指向加解密双方的共享数据 1，用于 IES 中的 KDF 运算。
byteLen_SharedInfo2	unsigned int	1	表示加解密双方共享数据 2 的长度，单位是字节。
SharedInfo2	const void *	由输入确定	指向加解密双方的共享数据 2，用于 IES 中的 HMAC 运算。

3.1.2.4.5 ECC 参数合法性校验函数

函数接口	unsigned int HED_ECCCurveCheck(ECC_Init_Parameters *EccInitPara)
功能说明	校验外部输入的曲线参数的合法性
输入	ECC_Init_Parameters *EccInitPara：其成员指向运算中输入输出数据的地址，在调用该函数时需要为如下成员分配地址： ➢ EccInitPara->iEccP：椭圆曲线的参数 P； ➢ EccInitPara->iEccA：椭圆曲线的参数 A； ➢ EccInitPara->iEccB：椭圆曲线的参数 B； ➢ EccInitPara->*iEccR：椭圆曲线的参数 N； ➢ EccInitPara->iEccG：椭圆曲线的参数 G； ➢ EccInitPara->bitLen：椭圆曲线位长； ➢ EccInitPara->hashType：进行 ECC 哈希运算的类型，本函数不用设置。
输出	无
返回值	RetVal：返回函数执行状态： ➢ 0x80A5：曲线参数正确； ➢ 其它值：曲线参数错误。
资源使用	栈使用约为 30 字节。
说明	➢ 在调用 ECC 参数初始化函数之前，使用此函数检查曲线参数的合法性。

3.1.2.4.6 ECC 参数初始化函数

函数接口	unsigned int HED_ECCInitPara(ECC_Init_Parameters *EccInitPara)
功能说明	由基本参数生成用于 ECC 其它函数运算的参数数据
输入	ECC_Init_Parameters *EccInitPara：其成员指向运算中输入输出数据的地址，在调用该函数时需要为如下成员分配地址：

仅用于华大内部开发使用

	➢ EccInitPara->iEccP：椭圆曲线的参数 P； ➢ EccInitPara->iEccA：椭圆曲线的参数 A； ➢ EccInitPara->iEccB：椭圆曲线的参数 B； ➢ EccInitPara->*iEccR：椭圆曲线的参数 N； ➢ EccInitPara->iEccG：椭圆曲线的参数 G； ➢ EccInitPara->bitLen：椭圆曲线位长； ➢ EccInitPara->hashType：进行 ECC 哈希运算的类型： ● 0x0001，sha-1； ● 0x0002，sha-224； ● 0x0003，sha-256； ● 0x0004，sha-384； ● 0x0005，sha-512； ● 0x00FF，内部根据位长设置。
输出	➢ EccInitPara->oDataPtr：输出处理好的 ECC 参数数据，长度为(bitLen/32)*14+8 个字。
返回值	RetVal：返回函数执行状态： ➢ 0x80A5：执行成功； ➢ 0x92C9：执行出错，位长无效或者对应的哈希函数不存在。
资源使用	栈使用约为 368 字节
说明	➢ 支持 192、224、256、384、512 位曲线参数设置； ➢ 对于非 32 位整数倍位长，返回值为 0x92C9； ➢ 对于输入的哈希函数类型，会检查当前库中是否存在对应的哈希函数，如果不存在，返回值为 0x92C9； ➢ 如果 lib 库提供了对应参数数据，建议优先选择库提供曲线参数数据，其性能高于由该函数处理的 ECC 曲线参数； ➢ 输出数据缓存需要在调用前预先分配，输入数据和输出数据空间不可以重合。

3.1.2.4.7 ECC 私钥校验函数

函数接口	unsigned int HED_ECCIsLegalPriKey(ECC_Parameters *ECC_Param)
功能说明	根据输入的曲线参数，校验私钥是否合法
输入	ECC_Parameters *ECC_Param：其成员指向运算中输入输出数据的地址，在调用该函数时需要为如下成员分配地址： ➢ ECC_Param->priKeyPtr：ECC_Param->priKeyPtr：指向用户私钥，长度为 N 字，私钥已经写入； ➢ ECC_Param->argPtr：指向椭圆曲线参数地址，曲线参数采取特定格式组织。
输出	无
返回值	RetVal：返回函数执行状态： ➢ 0x80A5：校验成功；

仅用于华大电子开发使用

	➢ 0x91B7：曲线参数无效； ➢ 0x92C9：校验失败。
资源使用	栈使用约为 36 字节
说明	➢ 函数中使用密钥均为不足整字时，高位补零凑齐整字； ➢ 函数中使用的曲线参数以数组形式与 API 一同提供或者通过 HED_ECCInitPara 函数生成，使用时，只需要把对应曲线参数的首地址传入即可。

3.1.2.4.8 ECC 公钥校验函数

函数接口	unsigned int HED_ECCIsLegalPubKey(ECC_Parameters *ECC_Param)
功能说明	根据输入的曲线参数，校验公钥是否合法
输入	ECC_Parameters *ECC_Param：其成员指向运算中输入输出数据的地址，在调用该函数时需要为如下成员分配地址： ➢ ECC_Param->pubKeyPtr：指向用户私钥，长度为 N*2 字，公钥已经写入； ➢ ECC_Param->argPtr：指向椭圆曲线参数地址，曲线参数采取特定格式组织。
输出	无
返回值	RetVal：返回函数执行状态： ➢ 0x80A5：校验成功； ➢ 0x91B7：曲线参数无效； ➢ 0x92C9：校验失败。
资源使用	栈使用约为 128 字节
说明	➢ 函数中使用密钥均为不足整字时，高位补零凑齐整字。 ➢ 函数中使用的曲线参数以数组形式与 API 一同提供或者通过 HED_ECCInitPara 函数生成，使用时，只需要把对应曲线参数的首地址传入即可。

3.1.2.4.9 ECC 密钥生成

函数接口	unsigned int HED_ECCGenKey(ECC_Parameters *ECC_Param) unsigned int HED_ECCGenKey2(ECC_Parameters *ECC_Param)
功能说明	密钥生成
输入	ECC_Parameters *ECC_Param：其成员指向运算中输入输出数据的地址，在调用该函数时需要为如下成员分配地址： ➢ ECC_Param->priKeyPtr：指向分配好的一块内存，内存大小等于生成的私钥长度； ➢ ECC_Param->pubKeyPtr：指向分配好的的一块内存，内存大小等于生成的公钥长度； ➢ ECC_Param->rndPtr：只在 HED_ECCGenKey2 中使用，指向生成私钥的随机数，长度为 N 字，随机数已写入； ➢ ECC_Param->argPtr：指向椭圆曲线参数地址，曲线参数采取特定格式

仅用于华大信开发使用

	组织。
输出	➢ 私钥：私钥长度为 N 字，输出到 ECC_Param->priKeyPtr 指向的内存中； ➢ 公钥：公钥长度为 2*N 字，输出到 ECC_Param->pubKeyPtr 指向的内存中，其中前 N 字为公钥 X 项，后 N 字为 Y 项。
返回值	RetVal：返回函数执行状态： ➢ 0x80A5：执行成功； ➢ 0x91B7：曲线参数无效； ➢ 0x92C9：生成密钥失败（随机数发生器失效，只在 HED_ECCGenKey 中使用）； ➢ 0x93DB：随机数无效（只在 HED_ECCGenKey2 中使用）。
资源使用	栈使用约为 360 字节
说明	➢ HED_ECCGenKey 内部调用随机数函数，生成私钥和公钥； ➢ HED_ECCGenKey2 由用户从外部输入随机数，利用该随机数产生私钥和公钥，外部输入随机数时，需要保证输入随机数的合法性，字长与 N 相同，数值位于[1,n-1]区间，n 为曲线参数； ➢ 如果 HED_ECCGenKey2 返回值为 0x93DB，本次输入的随机数无效，需要用户重新输入随机数，再次调用本函数； ➢ 函数中使用的随机数、生成的密钥均为不足整字时，高位补零凑齐整字； ➢ 函数中使用的曲线参数以数组形式与 API 一同提供或者通过 HED_ECCInitPara 函数生成，使用时，只需要把对应曲线参数的首地址传入即可。

3.1.2.4.10 ECC 签名运算

函数接口	unsigned int HED_ECCSign(ECC_Parameters *ECC_Param)
功能说明	签名运算
输入	ECC_Parameters *ECC_Param：其成员指向运算中输入输出数据的地址，在调用该函数时需要为如下成员分配地址： ➢ ECC_Param->iDataPtr：指向消息摘要，取决于使用的摘要算法长度 N_{Hash}，具体见下面说明； ➢ ECC_Param->rndPtr：保留； ➢ ECC_Param->priKeyPtr：指向用户私钥，长度为 N 字，私钥已经写入； ➢ ECC_Param->oDataPtr：指向分配好的的一块内存，内存大小不小于输出结果长度； ➢ ECC_Param->argPtr：指向椭圆曲线参数地址，曲线参数采取特定格式组织。
输出	➢ 签名结果输出到 signPara->oDataPtr 指向的内存中，长度等于对应曲线参数字节长度的两倍，具体见下面说明； ➢ 参数校验值，输出到 ECC_Param->paraChkSum 中，其值为(unsigned int)(ECC_Param->iDataPtr)+ (unsigned int)(ECC_Param->oDataPtr)+ (unsigned int)(ECC_Param->priKeyPtr)。
返回值	RetVal：返回函数执行状态：

仅用于东信项目使用

	➢ 0x80A5：签名成功； ➢ 0x91B7：曲线参数无效； ➢ 0x92C9：签名失败；
资源使用	栈使用约为 800 字节
说明	➢ HED_ECCSign 内部调用随机数函数，生成签名中的随机数； ➢ 输入输出数据地址不可以重合； ➢ 签名中采用的摘要算法除了 192、521 位，其余对应相应位长的 sha 算法：对于 P-192，采用 SHA-1，消息摘要长度N_{Hash}为 20 字节；对于 P-256，采用 SHA256，消息摘要长度N_{Hash}为 32 字节；对于 P-521，采用 SHA512，消息摘要长度N_{Hash}为 64 字节； ➢ 签名值数据格式符合 EMVCo 相关标准，结构为 r||s，长度为曲线参数字节长度的两倍：对于 P-256，签名值长度为 64 字节；对于 P-521，签名值长度为 132 字节； ➢ 函数中使用的随机数、密钥均为不足整字时，高位补零凑齐整字； ➢ 函数中使用的曲线参数以数组形式与 API 一同提供或者通过 HED_ECCInitPara 函数生成，使用时，只需要把对应曲线参数的首地址传入即可； ➢ 为了防止故障攻击，用户需要检查函数返回值及参数校验值是否正确。

3.1.2.4.11 ECC 签名验证运算

函数接口	unsigned int HED_ECCVerify(ECC_Parameters *ECC_Param)
功能说明	验证运算
输入	ECC_Parameters *ECC_Param：其成员指向运算中输入输出数据的地址，在调用该函数时需要为如下成员分配地址： ➢ ECC_Param->iDataPtr：指向消息摘要，取决于使用的摘要算法长度N_{Hash}，具体见下面说明； ➢ ECC_Param->oDataPtr：指向签名值，签名值已写入，格式见下面说明； ➢ ECC_Param-> pubKeyPtr：指向用户公钥，长度为 2*N 字，公钥已经写入； ➢ ECC_Param->argPtr：指向椭圆曲线参数地址，曲线参数采取特定格式组织。
输出	无
返回值	RetVal：返回函数执行状态： ➢ 0x80A5：验证成功； ➢ 0x91B7：曲线参数无效； ➢ 0x92C9：验证失败。
资源使用	栈使用约为 512 字节
说明	➢ 签名中采用的摘要算法除了 192、521 位，其余对应相应位长的 sha 算法：对于 P-192，采用 SHA-1，消息摘要长度N_{Hash}为 20 字节；对于 P-256，采用 SHA256，消息摘要长度N_{Hash}为 32 字节；对于 P-521，采用 SHA512，消息摘要长度N_{Hash}为 64 字节；

仅用于芯片开发使用

	➢ 签名值数据格式符合 EMVCo 相关标准，结构为 r||s，长度为曲线参数字节长度的两倍：对于 P-224，签名值长度为 56 字节；对于 P-256，签名值长度为 64 字节；对于 P-521，签名值长度为 132 字节； ➢ 函数中使用的随机数、密钥均为不足整字时，高位补零凑齐整字； ➢ 函数中使用的曲线参数以数组形式与 API 一同提供或者通过 HED_ECCInitPara 函数生成，使用时，只需要把对应曲线参数的首地址传入即可。

3.1.2.4.12 EMVCO ECC 加密运算

函数接口	unsigned int HED_ECCEncrypt(ECC_Parameters *ECC_Param, unsigned int iLen)
功能说明	加密运算
输入	ECC_Parameters *ECC_Param：其成员指向运算中输入输出数据的地址，在调用该函数时需要为如下成员分配地址： ➢ ECC_Param->iDataPtr：指向待加密的消息，长度为 iLen 字节，消息已写入； ➢ ECC_Param->oDataPtr：指向分配好的一块内存，长度由输出数据长度确定； ➢ ECC_Param->rndPtr：保留； ➢ ECC_Param-> pubKeyPtr：指向用户公钥，长度为 2*N 字，公钥已经写入； ➢ ECC_Param->argPtr：指向椭圆曲线参数地址，曲线参数采取特定格式组织； ➢ iLen：输入消息的字节长度。
输出	➢ 加密值，输出到 ECC_Param->oDataPt 指向的内存中，表示为 R||X||T，长度为三部分长度之和，具体见下面说明。
返回值	RetVal：返回函数执行状态： ➢ 0x80A5：加密成功； ➢ 0x91B7：曲线参数无效； ➢ 0x92C9：加密失败（随机数生成器失效，只在 HED_ECCEncrypt 中使用；输入数据长度为零；加密运算中出现不合法的无穷远点）。
资源使用	栈使用约为 856 字节
说明	➢ HED_ECCEncrypt 内部调用随机数函数，生成加密中的随机数； ➢ 入输出数据地址不可以重合； ➢ 加密中采用的摘要算法除了 192 位，其余对应相应位长的 sha 算法：对于 P-192，采用 SHA-1，消息摘要长度N_{Hash}为 20 字节；对于 P-224，采用 SHA224，消息摘要长度N_{Hash}为 28 字节； ➢ 加密输出数据格式符合 EMVCo 相关标准，结构为 R||X||T，长度为$2\lceil(\log_2 p)/8\rceil + iLen + {}^{N_{Hash}}/_{2}$（p 为曲线参数），即曲线参数字节长度的两倍+输入消息长度+消息摘要长度的一半：对于 P-256，加密数据长度为

仅用于华信开发使用

	iLen+80；对于 P-521，加密数据长度为 iLen+164； ➢ 函数中使用的随机数、密钥均为不足整字时，高位补零凑齐整字； ➢ 函数中使用的曲线参数以数组形式与 API 一同提供或者通过 HED_ECCInitPara 函数生成，使用时，只需要把对应曲线参数的首地址传入即可。

3.1.2.4.13 EMVCO ECC 解密运算

函数接口	unsigned int HED_ECCDecrypt(ECC_Parameters *ECC_Param, unsigned int iLen)
功能说明	解密运算
输入	ECC_Parameters *ECC_Param：其成员指向运算中输入输出数据的地址，在调用该函数时需要为如下成员分配地址： ➢ ECC_Param->iDataPtr：指向密文，长度为 iLen 字节，密文已写入，表示为 R||X||T； ➢ ECC_Param->oDataPtr：指向分配好的一块内存，长度由输出数据长度确定； ➢ ECC_Param-> priKeyPtr：指向用户私钥，长度为 N 字，私钥已经写入； ➢ ECC_Param->argPtr：指向椭圆曲线参数地址，曲线参数采取特定格式组织； ➢ iLen：密文字节长度。
输出	➢ 解密值，数据输出到 ECC_Param->oDataPtr 指向的内存中，具体内容见下面说明； ➢ 参数校验值，输出到 ECC_Param->paraChkSum 中，其值为(unsigned int)(ECC_Param->iDataPtr)+ (unsigned int)(ECC_Param->oDataPtr)+ (unsigned int)(ECC_Param->priKeyPtr)。
返回值	RetVal：返回函数执行状态： ➢ 0x80A5：解密成功； ➢ 0x91B7：曲线参数无效； ➢ 0x92C9：解密失败（输入数据不正确，长度不合法）。
资源使用	栈使用约为 900 字节
说明	➢ 输入输出数据地址不可以重合； ➢ 加密中采用的摘要算法除了 192 位，其余对应相应位长的 sha 算法：对于 P-192，采用 SHA-1，消息摘要长度N_{Hash}为 20 字节；对于 P-224，采用 SHA224，消息摘要长度N_{Hash}为 28 字节； ➢ 密文数据格式符合 EMVCo 相关标准，结构为 R||X||T，本函数参数输入 iLen 为密文数据长度，输出解密数据长度计算方法参考 ECC 加密函数说明中相关描述：对于 P-256，解密数据长度为 iLen-80；对于 P-521，解密数据长度为 iLen-164； ➢ 函数中使用的随机数、密钥均为不足整字时，高位补零凑齐整字； ➢ 函数中使用的曲线参数以数组形式与 API 一同提供或者通过 HED_ECCInitPara 函数生成，使用时，只需要把对应曲线参数的首地址传入即可；

仅用于华信开发使用

	➢ 为了防止故障攻击，用户需要检查函数返回值及参数校验值是否正确。

3.1.2.4.14 SEC ECC 加密运算

函数接口	unsigned int HED_ECCEncrypt_SEC(ECC_Parameters *ECC_Param, ECIES_Parameters *IES_Param, unsigned int iLen)
功能说明	加密运算
输入	ECC_Parameters *ECC_Param：其成员指向运算中输入输出数据的地址，在调用该函数时需要为如下成员分配地址： ➢ ECC_Param->iDataPtr：指向待加密的消息，长度为 iLen 字节，消息已写入； ➢ ECC_Param->oDataPtr：指向分配好的一块内存，长度由输出数据长度确定； ➢ ECC_Param->rndPtr：保留； ➢ ECC_Param-> pubKeyPtr：解密方临时公钥数据存放地址，长度为 2*N 字，公钥已经写入； ➢ ECC_Param->argPtr：指向椭圆曲线参数地址，曲线参数采取特定格式组织； ECIES_Parameters *IES_Param：其成员指向 IES 运算中输入数据的地址或长度等，在调用该函数时需要为如下成员赋值： ➢ IES_Param->byteLen_SharedInfo1：加解密双方共享数据 1 的长度； ➢ IES_Param->SharedInfo1：加解密双方共享数据 1 的存放地址； ➢ IES_Param->byteLen_SharedInfo2：加解密双方共享数据 2 的长度； ➢ IES_Param->SharedInfo2：加解密双方共享数据 2 的存放地址； ➢ iLen：输入待加密消息的字节长度。
输出	➢ 加密值，输出到 ECC_Param->oDataPt 指向的内存中，表示为 R||EM||D，长度为三部分长度之和，具体见下面说明。
返回值	RetVal：返回函数执行状态： ➢ 0x80A5：加密成功 ➢ 0x91B7：曲线参数无效 ➢ 0x92C9：加密失败（随机数生成器失效，只在 HED_ECCEncrypt 中使用；输入数据长度为零；加密运算中出现不合法的无穷远点）
资源使用	栈使用约为 900 字节
说明	➢ HED_ECCEncrypt_SEC 内部调用随机数函数，生成加密中的随机数； ➢ 输入输出数据地址不可以重合； ➢ 加密输出数据格式符合 SEC 标准，结构为 R||EM||D，长度为：曲线参数字节长度的两倍+消息长度+消息摘要长度：对于 P-256，加密数据长度为 iLen+96； ➢ 函数中使用的随机数、密钥均为不足整字时，高位补零凑齐整字； ➢ 函数中使用的曲线参数以数组形式与 API 一同提供或者通过 HED_ECCInitPara 函数生成，使用时，只需要把对应曲线参数的首地址传入即可；

仅用于华大电子开发使用

3.1.2.4.15 SEC ECC 解密运算

函数接口	unsigned int HED_ECCDecrypt_SEC(ECC_Parameters *ECC_Param, ECIES_Parameters *IES_Param, unsigned int iLen)
功能说明	解密运算
输入	ECC_Parameters *ECC_Param：其成员指向运算中输入输出数据的地址，在调用该函数时需要为如下成员分配地址： ➢ ECC_Param->iDataPtr：指向内存中的密文，长度为 iLen 字节，密文已写入，表示为 R||EM||D； ➢ ECC_Param->oDataPtr：指向分配好的一块内存，长度是 iLen-96； ➢ ECC_Param->priKeyPtr：解密方临时私钥数据存放地址，长度为 N 字，私钥已经写入； ➢ ECC_Param->argPtr：指向椭圆曲线参数地址，曲线参数采取特定格式组织； ECIES_Parameters *IES_Param：其成员指向 IES 运算中输入数据的地址或长度等，在调用该函数时需要为如下成员赋值： ➢ IES_Param->byteLen_SharedInfo1：加解密双方共享数据 1 的长度； ➢ IES_Param->SharedInfo1：加解密双方共享数据 1 的存放地址； ➢ IES_Param->byteLen_SharedInfo2：加解密双方共享数据 2 的长度； ➢ IES_Param->SharedInfo2：加解密双方共享数据 2 的存放地址； ➢ iLen：密文字节长度。
输出	➢ 解密值，数据输出到 ECC_Param->oDataPtr 指向的内存中，具体内容见下面说明； ➢ 参数校验值，输出到 ECC_Param->paraChkSum 中，其值为(unsigned int)(ECC_Param->iDataPtr)+ (unsigned int)(ECC_Param->oDataPtr)+ (unsigned int)(ECC_Param->priKeyPtr)。
返回值	RetVal：返回函数执行状态 ➢ 0x80A5：解密成功； ➢ 0x91B7：曲线参数无效； ➢ 0x92C9：解密失败（输入数据不正确，长度不合法）。
资源使用	栈使用约为 900 字节
说明	➢ 输入输出数据地址不可以重合； ➢ 密文数据格式符合 SEC 标准，结构为 R||EM||D，长度为：曲线参数字节长度的两倍+明文消息长度+消息摘要长度； ➢ 函数中使用的随机数、密钥均为不足整字时，高位补零凑齐整字； ➢ 函数中使用的曲线参数以数组形式与 API 一同提供或者通过 HED_ECCInitPara 函数生成，使用时，只需要把对应曲线参数的首地址传入即可； ➢ 为了防止故障攻击，用户需要检查函数返回值及参数校验值是否正确。

仅用于华大信安开发使用

3.1.2.4.16 ECKA 运算函数

函数接口	unsigned int HED_ECKA_BSI(ECC_Parameters *ECC_Param)
功能说明	ECDH 运算
输入	ECC_Parameters *ECC_Param：其成员指向运算中输入输出数据的地址，在调用该函数时需要为如下成员分配地址： ➢ ECC_Param->iDataPtr：保留； ➢ ECC_Param->rndPtr：保留； ➢ ECC_Param->priKeyPtr：指向 A 方临时私钥； ➢ ECC_Param-> pubKeyPtr：指向对方临时公钥； ➢ ECC_Param->oDataPtr：指向分配好的的一块内存，内存大小不小于输出结果长度； ➢ ECC_Param->argPtr：指向椭圆曲线参数地址，曲线参数采取特定格式组织。
输出	➢ 结果输出到 ECC_Param->oDataPtr 指向的内存中，长度等于对应曲线参数字节长度的两倍，输出为S_{AB}，其中前半段为Z_{AB}； ➢ 参数校验值，输出到 ECC_Param->paraChkSum 中，其值为(unsigned int)(ECC_Param->pubKeyPtr)+ (unsigned int)(ECC_Param->oDataPtr)+ (unsigned int)(ECC_Param->priKeyPtr)
返回值	RetVal：返回函数执行状态： ➢ 0x80A5：执行成功； ➢ 0x91B7：曲线参数无效； ➢ 0x92C9：运算失败。
资源使用	栈使用约为 520 字节
说明	➢ 函数中使用的曲线参数以数组形式与 API 一同提供或者通过 HED_ECCIniPara 函数生成，使用时，只需要把对应曲线参数的首地址传入即可； ➢ ECKA 符合《BSI-TR-03111 Version 2.10》； ➢ 为了防止故障攻击，用户需要检查函数返回值及参数校验值是否正确。

3.1.2.4.17 KDF 运算函数

函数接口	typedef struct { unsigned char *pData; unsigned short __pad; unsigned short dataLen; } KDF_BigNum; /* ---------- end of struct BigNum ---------- */ void HED_KDF_X963(const KDF_BigNum *zAB, const KDF_BigNum *r, KDF_BigNum *key, unsigned int hashType)
功能说明	KDF 运算
输入	zAB 为运算中 zAB 数据信息： ➢ zAB->pData 指向 HED_ECKA_BSI 运算结果；

	➢ zAB->dataLen 为椭圆曲线字节长度，比如 256 位椭圆曲线，该值为 32； r 为运算中共享信息数据，如果没有共享数据，r 置为 NULL： ➢ r->pData 指向共享信息。 ➢ r->dataLen 为共享信息字节长度。 key 为共享密钥： ➢ key->pData 指向共享密钥输出地址。 ➢ key->dataLen 为共享密钥字节长度。 hashType：进行 KDF 运算使用的哈希函数类型： ● 0x0001，sha-1； ● 0x0002，sha-224； ● 0x0003，sha-256； ● 0x0004，sha-384； ● 0x0005，sha-512。
输出	➢ key->pData 指向输出共享密钥。
返回值	无
资源使用	栈使用约为 480 字节
说明	➢ 如果输入哈希类型不符合上述几种，将不执行 KDF 运算； ➢ KDF 函数符合 ANSI X9.63 [3]，与《BSI-TR-03111 Version 2.10》中描述一致。

3.1.2.4.18 ECC 点压缩函数

函数接口	void HED_ECCPointCompress(unsigned char *x_y, const unsigned int *point, unsigned int bitLen)
功能说明	点压缩
输入	➢ point：指向点，长度为 2*N 字，格式为 x||y； ➢ bitLen：椭圆曲线位长。
输出	➢ x_y：输出点压缩后的值，前面一字节表征 y 坐标的最低位值，后面为 x 的值。
返回值	无
资源使用	栈使用约为 48 字节
说明	➢ 输出 x 值的字节数为$\lceil bitLen/8 \rceil$； ➢ 该函数不支持 nistp224 和 segg_prime_224k1、nistp521 曲线。

3.1.2.4.19 ECC 点解压缩函数

函数接口	unsigned int HED_ECCPointDecompress(unsigned int *point, const unsigned char *x_y, const unsigned int *argPtr)
功能说明	点解压缩
输入	➢ x_y：点压缩后的值，前面一字节 y 坐标的最低位值，后面为 x 值；

仅用于华信开发使用

	➢ argPtr：指向椭圆曲线参数地址，曲线参数采取特定格式组织。
输出	➢ point：指向解压缩后的点，长度为 2*N 字，格式为 x||y。
返回值	RetVal：返回函数执行状态： ➢ 0x80A5：成功； ➢ 0x91B7：参数错误； ➢ 0x92C9：不支持的曲线。
资源使用	栈使用约为 312 字节
说明	➢ 函数中使用的曲线参数以数组形式与 API 一同提供或者通过 HED_ECCInitPara 函数生成，使用时，只需要把对应曲线参数的首地址传入即可； ➢ 该函数不支持 nistp224 和 segg_prime_224k1、nistp521 曲线。

3.1.2.4.20 EC-SDSA 签名运算

函数接口	unsigned int HED_ECCSSign(ECC_Parameters *ECC_Param,unsigned int iLen)
功能说明	EC-SDSA 签名运算
输入	ECC_Parameters *ECC_Param：其成员指向运算中输入输出数据的地址，在调用该函数时需要为如下成员分配地址： ➢ ECC_Param->iDataPtr：指向消息摘要，取决于使用的摘要算法长度 N_{Hash}，具体见下面说明； ➢ ECC_Param->rndPtr：保留； ➢ ECC_Param->priKeyPtr：指向用户私钥，长度为 N 字，私钥已经写入； ➢ ECC_Param->oDataPtr：指向分配好的的一块内存，内存大小不小于输出结果长度； ➢ ECC_Param->argPtr：指向椭圆曲线参数地址，曲线参数采取特定格式组织。 ➢ iLen：输入待签名消息的字节长度。
输出	➢ 签名结果输出到 signPara->oDataPtr 指向的内存中，长度等于对应 $N_{Hash} + N_{FIELD}$ 具体见下面说明； ➢ 参数校验值，输出到 ECC_Param->paraChkSum 中，其值为(unsigned int)(ECC_Param->iDataPtr)+ (unsigned int)(ECC_Param->oDataPtr)+ (unsigned int)(ECC_Param->priKeyPtr)。
返回值	RetVal：返回函数执行状态： ➢ 0x80A5：签名成功； ➢ 0x91B7：曲线参数无效； ➢ 0x92C9：签名失败；
资源使用	栈使用约为 800 字节
说明	➢ HED_ECCSign 内部调用随机数函数，生成签名中的随机数； ➢ 输入输出数据地址不可以重合； ➢ 签名中采用的摘要算法除了 192、521 位，其余对应相应位长的 sha 算法：对于 P-192，采用 SHA-1，消息摘要长度 N_{Hash} 为 20 字节；对于 P-256，采用 SHA256，消息摘要长度 N_{Hash} 为 32 字节；对于 P-521，采用 SHA512，消息摘要长度 N_{Hash} 为 64 字节，N_{FIELD} 为椭圆曲线字节长度；

仅限于东信中发使用

	➢ 签名值数据格式符合 EMVCo 相关标准，结构为 r||s，长度为$N_{Hash}+N_{FIELD}$：对于 P-256，采用 SHA256 签名值长度为 64 字节；对于 P-521,，采用 SHA512 签名值长度为 130 字节； ➢ 函数中使用的随机数、密钥均为不足整字时，高位补零凑齐整字； ➢ 函数中使用的曲线参数以数组形式与 API 一同提供或者通过 HED_ECCInitPara 函数生成，使用时，只需要把对应曲线参数的首地址传入即可； ➢ 为了防止故障攻击，用户需要检查函数返回值及参数校验值是否正确。

3.1.2.4.21 EC-SDSA 签名验证运算

函数接口	unsigned int HED_ECCSVerify(ECC_Parameters *ECC_Param,unsigned int iLen)
功能说明	EC-SDSA 验证运算
输入	ECC_Parameters *ECC_Param：其成员指向运算中输入输出数据的地址，在调用该函数时需要为如下成员分配地址： ➢ ECC_Param->iDataPtr：指向消息摘要，取决于使用的摘要算法长度N_{Hash}，具体见下面说明； ➢ ECC_Param->oDataPtr：指向签名值，签名值已写入，格式见下面说明； ➢ ECC_Param-> pubKeyPtr：指向用户公钥，长度为 2*N 字，公钥已经写入； ➢ ECC_Param->argPtr：指向椭圆曲线参数地址，曲线参数采取特定格式组织； ➢ iLen：输入待签名消息的字节长度。
输出	无
返回值	RetVal：返回函数执行状态： ➢ 0x80A5：验证成功； ➢ 0x91B7：曲线参数无效； ➢ 0x92C9：验证失败。
资源使用	栈使用约为 512 字节
说明	➢ 签名中采用的摘要算法除了 192、521 位，其余对应相应位长的 sha 算法：对于 P-192，采用 SHA-1，消息摘要长度N_{Hash}为 20 字节；对于 P-256，采用 SHA256，消息摘要长度N_{Hash}为 32 字节；对于 P-521，采用 SHA512，消息摘要长度N_{Hash}为 64 字节； ➢ 签名值数据格式符合 EMVCo 相关标准，结构为 r||s，长度为$N_{Hash}+N_{FIELD}$：对于 P-256，采用 SHA256 签名值长度为 64 字节；对于 P-521,，采用 SHA512 签名值长度为 130 字节； ➢ 函数中使用的随机数、密钥均为不足整字时，高位补零凑齐整字； ➢ 函数中使用的曲线参数以数组形式与 API 一同提供或者通过 HED_ECCInitPara 函数生成，使用时，只需要把对应曲线参数的首地址传入即可。

3.1.2.4.22 EC-BDH 运算函数

函数接口	unsigned int HED_ECCBdh(ECC_Parameters* ECC_Param);

华大电子开发使用

功能说明	计算 BDH 共享密钥
输入	ECC_Parameters *ECC_Param：其成员指向运算中输入输出数据的地址，在调用该函数时需要为如下成员分配地址： ➢ ECC_Param->iDataPtr：保留； ➢ ECC_Param->rndPtr：指向盲化因子 r； ➢ ECC_Param->priKeyPtr：指向 A 方临时私钥； ➢ ECC_Param-> pubKeyPtr：指向对方临时公钥； ➢ ECC_Param->oDataPtr：指向分配好的的一块内存，内存大小不小于输出结果长度； ➢ ECC_Param->argPtr：指向椭圆曲线参数地址，曲线参数采取特定格式组织。
输出	➢ 结果输出到 ECC_Param->oDataPtr 指向的内存中，长度等于对应曲线参数字节长度的两倍，输出为S_{AB}，其中前半段为Z_{AB}； ➢ 参数校验值，输出到 ECC_Param->paraChkSum 中，其值为(unsigned int)(ECC_Param->pubKeyPtr)+ (unsigned int)(ECC_Param->oDataPtr)+ (unsigned int)(ECC_Param->priKeyPtr)
返回值	RetVal：返回函数执行状态： ➢ 0x80A5：执行成功； ➢ 0x91B7：曲线参数无效； ➢ 0x92C9：运算失败。
资源使用	栈使用约为 520 字节
说明	➢ 函数中使用的曲线参数以数组形式与 API 一同提供或者通过 HED_ECCInitPara 函数生成，使用时，只需要把对应曲线参数的首地址传入即可； BDH符合《EMV Contactless Book E Security and Key Management v1.0》； ➢ 为了防止故障攻击，用户需要检查函数返回值及参数校验值是否正确。

3.1.2.5 SHA-1 算法 API

3.1.2.5.1 功能介绍

本 API 用于 SHA-1 HASH 运算。

- **数据存储顺序**：哈希运算中用到的参数和数据，均为大端模式存储；
- **算法使用流程**：使用算法时必须按照如下顺序进行函数的调用：运算初始化->[Hash 运算非最后一块]->Hash 运算最后一块。其中，非最后一块运算的数据长度必须是 64 字节，最后一块数据长度为 0-63 字节。最后一块运算时进行 message padding 操作。

3.1.2.5.2 SHA-1 运算初始化

函数接口	void HED_SHA1_Init(unsigned int *V)
输入	➢ V：长度为 6 的无符号 32 位整型数组（24 字节），哈希计算初始值数据起

仅用于内部开发使用

	始地址。
输出	➢ V：长度为 6 的无符号 32 位整型数组（24 字节），哈希计算初始值数据起始地址。
返回值	无
资源使用	----
说明	➢ 函数将参数 v 指向地址的 24 字节长的数据进行初始化，用于后续哈希运算； ➢ 24 字节中，前 20 字节是哈希计算初始值，后 4 字节用于 API 记录 message 总长度，本 API 不支持 SHA-1 数据总长度超过 4G Bytes 的运算。

3.1.2.5.3 SHA-1 运算

函数接口	unsigned int HED_SHA1_Block(unsigned int *V, const void *DataValue, unsigned int DataLen)
输入	➢ V：初始值指针或前一轮运算结果，总长度 24 字节； ➢ DataValue：输入数据的指针； ➢ DataLen：输入数据长度，取值范围 0-64。
输出	➢ V：运算结果，长度为 6 的无符号 32 位整型数组（24 字节，前 20 字节是哈希计算中间结果，后 4 字节用于 API 记录 message 总长度）。
返回值	➢ 0： 运算完成； ➢ 1： 输入数据长度不正确。
资源使用	----
说明	➢ 第一次调用此函数前应执行 SHA-1 运算初始化； ➢ 当参与 Hash 运算的数据超过 64 个字节时，需要将数据按照 64 字节长度截取为多个数据块，最后一个数据块可以不足 64 个字节，然后顺序将每个数据块作为输入调用本函数进行运算； ➢ 如果用户的数据长度为 64 字节的整数倍，则最后需要以数据长度为零，调用一次该函数，才能获得最后的哈希值； ➢ 只有最后一块的长度才可以为 0-63，非最后一块的长度必须为 64 字节； ➢ 本 API 不支持 SHA-1 运算数据总长度超过（4G-8）bits 的运算的情况。

3.1.2.6 SHA-224 算法 API

3.1.2.6.1 功能介绍

本 API 用于 SHA-224 HASH 运算。

- **数据存储顺序**：哈希运算中用到的参数和数据，均为大端模式存储；
- **算法使用流程**：使用算法时必须按照如下顺序进行函数的调用：运算初始化->[Hash 运算非最后一块]->Hash 运算最后一块。其中，非最后一块运算的数据长度必须是 64 字节，最后一块数据长度为 0-63 字节。最后一块运算时进行 message padding 操作。

仅用于华大电子开发使用

3.1.2.6.2 SHA-224 运算初始化

函数接口	void HED_SHA224_Init(unsigned int *V)
输入	➢ V：长度为 9 的无符号 32 位整型数组（36 字节），哈希计算初始值数据起始地址。
输出	➢ V：长度为 9 的无符号 32 位整型数组（36 字节），哈希计算初始值数据起始地址。
返回值	无
资源使用	----
说明	➢ 函数将参数 v 指向地址的 36 字节长的数据进行初始化，用于后续哈希运算； ➢ 36 字节中，前 32 字节是哈希计算初始值，后 4 字节用于 API 记录 message 总长度； ➢ 本 API 不支持 SHA-224 运算数据总长度超过 4G Bytes 的情况。

3.1.2.6.3 SHA-224 运算

函数接口	unsigned int HED_SHA224_Block(unsigned int *V, const unsigned int *DataValue, unsigned int DataLen)
输入	➢ V：初始值指针或前一轮运算结果，总长度 36 字节； ➢ DataValue：输入数据的指针； ➢ DataLen：输入数据长度，取值范围 0-64。
输出	➢ V：运算结果，长度为 9 的无符号 32 位整型数组（36 字节，前 32 字节是哈希计算最终结果，后 4 字节用于 API 记录 message 总长度）。
返回值	➢ 0：运算完成； ➢ 1：输入数据长度不正确。
资源使用	----
说明	➢ 第一次调用此函数前应执行 SHA-224 运算初始化； ➢ 当参与 Hash 运算的数据超过 64 个字节时，需要将数据按照 64 字节长度截取为多个数据块，最后一个数据块可以不足 64 个字节，然后顺序将每个数据块作为输入调用本函数进行运算； ➢ 如果用户的数据长度为 64 字节的整数倍，则最后需要以数据长度为零，调用一次该函数，才能获得最后的哈希值。最后的哈希值有效长度为 28 字节，位于 32 字节计算结果的前 28 字节； ➢ 只有最后一块的长度才可以为 0-63，非最后一块的长度必须为 64 字节； ➢ 本 API 不支持 SHA-224 运算数据总长度超过（4G-8）bits 的运算的情况。

3.1.2.7 SHA-256 算法 API

3.1.2.7.1 功能介绍

本 API 用于 SHA-256 HASH 运算。

仅用于客户开发使用

- **数据存储顺序**：哈希运算中用到的参数和数据，均为大端模式存储；
- **算法使用流程**：使用算法时必须按照如下顺序进行函数的调用：运算初始化->[Hash 运算非最后一块]->Hash 运算最后一块。其中，非最后一块运算的数据长度必须是 64 字节，最后一块数据长度为 0-63 字节。最后一块运算时进行 message padding 操作。

3.1.2.7.2 SHA-256 运算初始化

函数接口	void HED_SHA256_Init(unsigned int *V)
输入	➢ V：长度为 9 的无符号 32 位整型数组（36 字节），哈希计算初始值数据起始地址。
输出	➢ V：长度为 9 的无符号 32 位整型数组（36 字节），哈希计算初始值数据起始地址。
返回值	无
资源使用	----
说明	➢ 函数将参数 v 指向地址的 36 字节长的数据进行初始化，用于后续哈希运算； ➢ 36 字节中，前 32 字节是哈希计算初始值，后 4 字节用于 API 记录 message 总长度； ➢ 本 API 不支持 SHA-256 运算数据总长度超过 4G Bytes 的情况。

3.1.2.7.3 SHA-256 运算

函数接口	unsigned int HED_SHA256_Block(unsigned int *V, const void *DataValue, unsigned int DataLen)
输入	➢ V：初始值指针或前一轮运算结果，总长度 36 字节； ➢ DataValue：输入数据的指针； ➢ DataLen：输入数据长度，取值范围 0-64。
输出	➢ V：运算结果，长度为 9 的无符号 32 位整型数组（36 字节，前 32 字节是哈希计算最终结果，后 4 字节用于 API 记录 message 总长度）。
返回值	➢ 0： 运算完成 ➢ 1： 输入数据长度不正确
资源使用	----
说明	➢ 第一次调用此函数前应执行 SHA-256 运算初始化； ➢ 当参与 Hash 运算的数据超过 64 个字节时，需要将数据按照 64 字节长度截取为多个数据块，最后一个数据块可以不足 64 个字节，然后顺序将每个数据块作为输入调用本函数进行运算； ➢ 如果用户的数据长度为 64 字节的整数倍，则最后需要以数据长度为零，调用一次该函数，才能获得最后的哈希值； ➢ 只有最后一块的长度才可以为 0-63，非最后一块的长度必须为 64 字节； ➢ 本 API 不支持 SHA-256 运算数据总长度超过（4G-8）bits 的运算的情况。

仅用于东信开发使用

3.1.2.8 SHA-384 算法 API

3.1.2.8.1 功能介绍

本 API 用于 SHA-384 HASH 运算。

- **数据存储顺序**：哈希运算中用到的参数和数据，均为大端模式存储；
- **算法使用流程**：使用算法时必须按照如下顺序进行函数的调用：运算初始化->[Hash 运算非最后一块]->Hash 运算最后一块。其中，非最后一块运算的数据长度必须是 128 字节，最后一块数据长度为 0-127 字节。最后一块运算时进行 message padding 操作。

3.1.2.8.2 SHA-384 运算初始化

函数接口	void HED_SHA384_Init(unsigned int *V)
输入	➢ V：长度为 18 的无符号 32 位整型数组（72 字节），哈希计算初始值数据起始地址。
输出	➢ V：长度为 18 的无符号 32 位整型数组（72 字节），哈希计算初始值数据起始地址。
返回值	无
资源使用	----
说明	➢ 函数将参数 v 指向地址的 72 字节长的数据进行初始化，用于后续哈希运算； ➢ 72 字节中，前 64 字节是哈希计算初始值，后 8 字节用于 API 记录 message 总长度； ➢ 本 API 不支持 SHA-384 运算数据总长度超过 4G Bytes 的情况。

3.1.2.8.3 SHA-384 运算

函数接口	unsigned int HED_SHA384_Block(unsigned int *V, const void *DataValue, unsigned int DataLen)
输入	➢ V：初始值指针或前一轮运算结果，长度为 18 的无符号 32 位整型数组（72 字节）； ➢ DataValue：输入数据的指针； ➢ DataLen：输入数据长度，取值范围 0-128。
输出	➢ V：运算结果，长度为 18 的无符号 32 位整型数组（72 字节，前 64 字节是哈希计算最终结果，后 8 字节用于 API 记录 message 总长度），最后块计算完毕后，只有前 12 个字有效。
返回值	➢ 0： 运算完成； ➢ 1： 输入数据长度不正确。
资源使用	----
说明	➢ 第一次调用此函数前应执行 SHA-384 运算初始化；

仅用于测试开发使用

	➢ 当参与 Hash 运算的数据超过 128 个字节时，需要将数据按照 128 字节长度截取为多个数据块，最后一个数据块可以不足 128 个字节，然后顺序将每个数据块作为输入调用本函数进行运算； ➢ 如果用户的数据长度为 128 字节的整数倍，则最后需要以数据长度为零，调用一次该函数，才能获得最后的哈希值； ➢ 只有最后一块的长度才可以为0-127,非最后一块的长度必须为128字节； ➢ 本 API 不支持 SHA-384 运算数据总长度超过（4G-8）bits 的运算的情况。

3.1.2.9 SHA-512 算法 API

3.1.2.9.1 功能介绍

本 API 用于 SHA-512 HASH 运算。

- **数据存储顺序**：哈希运算中用到的参数和数据，均为大端模式存储；
- **算法使用流程**：使用算法时必须按照如下顺序进行函数的调用：运算初始化->[Hash 运算非最后一块]->Hash 运算最后一块。其中，非最后一块运算的数据长度必须是 128 字节，最后一块数据长度为 0-127 字节。最后一块运算时进行 message padding 操作。

3.1.2.9.2 SHA-512 运算初始化

函数接口	void HED_SHA512_Init(unsigned int *V)
输入	➢ V：长度为 18 的无符号 32 位整型数组（72 字节），哈希计算初始值数据起始地址。
输出	➢ V：长度为 18 的无符号 32 位整型数组（72 字节），哈希计算初始值数据起始地址。
返回值	无
资源使用	----
说明	➢ 函数将参数 v 指向地址的 72 字节长的数据进行初始化，用于后续哈希运算； ➢ 72 字节中，前 64 字节是哈希计算初始值，后 8 字节用于 API 记录 message 总长度； ➢ 本 API 不支持 SHA-512 运算数据总长度超过 4G Bytes 的情况。

3.1.2.9.3 SHA-512 运算

函数接口	unsigned int HED_SHA512_Block(unsigned int *V, const void *DataValue, unsigned int DataLen)
输入	➢ V：初始值指针或前一轮运算结果，长度为 18 的无符号 32 位整型数组（72 字节）； ➢ DataValue：输入数据的指针； ➢ DataLen：输入数据长度，取值范围 0-128。

使用与发信开发使用

输出	➢ V：运算结果，长度为 18 的无符号 32 位整型数组（72 字节，前 64 字节是哈希计算最终结果，后 8 字节用于 API 记录 message 总长度）。
返回值	➢ 0： 运算完成； ➢ 1： 输入数据长度不正确。
资源使用	----
说明	➢ 第一次调用此函数前应执行 SHA-512 运算初始化； ➢ 当参与 Hash 运算的数据超过 128 个字节时，需要将数据按照 128 字节长度截取为多个数据块，最后一个数据块可以不足 128 个字节，然后顺序将每个数据块作为输入调用本函数进行运算； ➢ 如果用户的数据长度为 128 字节的整数倍，则最后需要以数据长度为零，调用一次该函数，才能获得最后的哈希值； ➢ 只有最后一块的长度才可以为0-127，非最后一块的长度必须为128字节； ➢ 本 API 不支持 SHA-512 运算数据总长度超过（4G-8）bits 的运算的情况。

3.1.2.10 X25519 算法 API

3.1.2.10.1 概述

本文档中描述的 X25519 接口基于 RFC 7748 实现。具体参见：

https://tools.ietf.org/html/rfc7748

https://www.rfc-Editor.org/errata_search.php?rfc=7748

3.1.2.10.2 X25519 函数库接口参数

X25519 函数的输入输出参数由结构体数据 X25519_Parameters 传递。X25519_Parameters 的定义如下：

```
typedef struct
{
    const unsigned char *priKeyPtr;
    const unsigned char *pubKeyPtr;
    unsigned char       *sharedKey;
    unsigned int        paraChkSum;
} X25519_Parameters;
```

表 3.3 成员变量说明

名称	类型	目标数据长度（字节长度）	描述
priKeyPtr	const unsigned char *	32	指向己方私钥。
pubKeyPtr	const unsigned char *	32	指向对方公钥。
sharedKey	unsigned char *	32	指向签名或解密中使用的随机数。
paraChkSum	unsigned int	4	输出函数参数校验值。

仅用于芯片开发使用

3.1.2.10.3 函数接口描述

函数接口	unsigned int HED_X25519Sec(X25519_Parameters *X25519_Para)
功能说明	X25519 运算运算
输入	➢ X25519_Para->priKeyPtr：己方私钥，32 字节，值已写入； ➢ X25519_Para->pubKeyPtr：对方公钥，32 字节，值已写入或者置位 NULL。
输出	➢ 输出公钥或者共享密钥，32 字节，输出到 X25519_Para->sharedKey； ➢ 参数校验值，输出到 X25519_Para->paraChkSum 中，其值为(unsigned int)(X25519_Para->priKeyPtr)+ (unsigned int)(X25519_Para->pubKeyPtr)+ (unsigned int)(X25519_Para->sharedKey)。
返回值	RetVal：返回函数执行状态： ➢ 0x80A5：执行成功； ➢ 0x92C9：执行失败；
资源使用	栈使用约为 300 字节
说明	➢ 如果 pubKeyPtr 输入 NULL，则函数内存会利用基点（值为 9）进行计算，结果为己方公钥； ➢ 如果 pubKeyPtr 输入 32 字节数据，则作为对方公钥，计算结果为共享密钥； ➢ 数据均为小端格式； ➢ 为了防止故障攻击，用户需要检查函数返回值及参数校验值是否正确。

3.1.2.10.4 函数流程说明

3.1.2.10.4.1X25519 接口实现 ECDH 流程

a) Alice 生成一个 32 字节的随机数，存放在 a[0]至 a[31]中，将 a 作为己方私钥，将对方公钥置位 NULL，调用 HED_X25519Sec 计算出公钥 K_A，将 K_A 发送给 Bob；

b) Bob 生成一个 32 字节的随机数，存放在 b[0]至 b[31]中，将 b 作为己方私钥，将对方公钥置位 NULL，调用 HED_X25519Sec 计算出公钥 K_**B**，将 K_B 发送给 Alice；

c) Alice 使用 a、KB 调用 HED_X25519Sec 计算出 shared_secret；

d) Bob 使用 b, K_A 调用 HED_X25519Sec 计算出 shared_secret。

3.1.2.10.4.2X25519 接口实现 ECDH 测试数据

Alice's private key, a：

77076d0a7318a57d3c16c17251b26645df4c2f87ebc0992ab177fba51db92c2a

Alice's public key, X25519(a, 9)：

8520f0098930a754748b7ddcb43ef75a0dbf3a0d26381af4eba4a98eaa9b4e6a

Bob's private key, b：

5dab087e624a8a4b79e17f8b83800ee66f3bb1292618b6fd1c2f8b27ff88e0eb

Bob's public key, X25519(b, 9)：

仅用于东信开发使用

de9Edb7d7b7dc1b4d35b61c2ece435373f8343c85b78674dadfc7e146f882b4f

Their shared secret, K：

4a5d9d5ba4ce2de1728e3bf480350f25e07e21c947d19e3376f09b3c1e161742

3.2 其他类 API

3.2.1 概述

目前此类 API 中只包含掩膜 API 的版本查询功能和功耗管理相关功能函数。

3.2.2 函数接口描述

3.2.2.1 芯片算法 API 区代码版本查询 API

3.2.2.1.1 功能介绍

本函数用于查询芯片算法 API 区的代码版本号。

版本格式说明：0xAABBMMXY，其中：

- 0xAA，表示芯片算法 API 区代码版本（HEX 版本），偏移地址为 0x00。其表示 0xBBMMXY 的总体变化，一般按大排行编制；
- 0xBB， RFU，固定为 00；
- 0xMM，表示芯片算法区非安全算法代码（除安全算法外的其他代码）的软件版本；
- 0xXY，表示芯片算法区安全算法代码（ECC/DES/TDES/AES）的软件版本 X.Y。一般 X 表示发布版本，Y 表示调试子版本。

3.2.2.1.2 接口定义

函数接口	unsigned int HED_ALGAPI_Version(void)
输入	无
输出	无
返回值	➢ bit31-bit24：芯片算法 API 区代码版本； ➢ bit23-bit16：RFU，0x00； ➢ bit15-bit8：芯片算法 API 区非安全算法代码版本； ➢ bit7-bit0：芯片算法 API 区安全算法代码版本。
资源使用	----
说明	无

3.2.2.2 时钟配置函数

3.2.2.2.1 函数说明

本函数用于实现相关的时钟调整操作，并开启时钟自适应功能。

仅用于华大信开发使用

3.2.2.2.2 函数接口

函数接口	void HED_ChangeSYSFreq(SYS_CLK_DIVISOR SysDivLmt, SYS_CLK_DIVISOR SysDivLowLmt, SYS_CLK_DIVISOR_LF SysDivLmt_lf, SYS_CLK_DIVISOR_LF SysDivLowLmt_lf)
输入	typedef enum { SYSCLK_5p5Mhz = 0x0, SYSCLK_11Mhz = 0x1, SYSCLK_16p5Mhz = 0x2, SYSCLK_22Mhz = 0x3, SYSCLK_27p5Mhz = 0x4, SYSCLK_33Mhz = 0x5, SYSCLK_38p5Mhz = 0x6, SYSCLK_44Mhz = 0x7, SYSCLK_49p5Mhz = 0x8, SYSCLK_55Mhz = 0x9, SYSCLK_60p5Mhz = 0xA, SYSCLK_66Mhz = 0xB, SYSCLK_71p5Mhz = 0xC SYSCLK_77Mhz = 0xD, SYSCLK_82p5Mhz = 0xE, SYSCLK_88Mhz = 0xF }SYS_CLK_DIVISOR; typedef enum { SYSCLK_LF_6p78Mhz = 0x1, SYSCLK_LF_10p17Mhz = 0x2, SYSCLK_LF_13p56Mhz = 0x3, }SYS_CLK_DIVISOR_LF;

仅用于东信开发使用

<table>
<tr><td></td><td>SysDiv：系统时钟（即 AHB 时钟）分频系数，取值为 SYS_CLK_DIVISOR 定义枚举值。
<table><tr><th>SysDiv</th><th>时钟大小</th></tr><tr><td>0x0~0xF</td><td>5.5*(SysDiv+1)MHz</td></tr></table>SYS_CLK_DIVISOR_LF 定义枚举值。
<table><tr><th>SysDiv_lf</th><th>时钟大小</th></tr><tr><td>0x1~0x3</td><td>3.39*(SysDiv + 1)MHz</td></tr></table>
SysDivLmt：接触或非接高场时钟上限
SysDivLowLmt：接触或非接高场时钟下限
SysDivLmt_lf：非接低场时钟上限
SysDivLowLmt_lf：非接低场时钟下限</td></tr>
<tr><td>输出</td><td>无</td></tr>
<tr><td>返回值</td><td>无</td></tr>
<tr><td>说明</td><td>➢ 当 SysDivLmt>= SysDivLowLmt，并且 SysDivLmt_lf>= SysDivLowLmt_lf 时，输入参数有效
➢ 无论输入参数是否有效，接口执行后自动开启时钟自适应功能</td></tr>
</table>

3.2.2.3 前置功耗管理策略函数

3.2.2.3.1 函数说明

本函数用于进入 standby 之前的功耗管理，如果用户调用 standby API，无需调用该函数，因为 standby API 中已经调用该函数。

3.2.2.3.2 接口定义

函数接口	void PreSet_SYSCLK_ICS (unsigned int PMStrategyIndex, unsigned int AlgTypeIndex,unsigned int * bakSFRBuf)
输入	PMStrategyIndex：表示前置功耗管理策略的索引 0x00：芯片进入 standby 之前的功耗策略

仅用于芯信开发使用

	其他：空操作 AlgTypeIndex： 本参数无意义
输出	bakSFRBuf：表示存储 3 个字长的 SFR 备份值的地址，两个 SFR 依次是 SYSCLKCFG、SYSICSCFG
返回值	无
资源使用	
说明	无

3.2.2.4 后置功耗管理策略函数

3.2.2.4.1 函数说明

本函数用于退出 standby 之后的功耗管理，如果用户调用 standby API，无需调用该函数，因为 standby API 中已经调用该函数。

3.2.2.4.2 接口定义

函数接口	void PostSet_SYSCLK_ICS (unsigned int PMStrategyIndex,unsigned int * bakSFRBuf)
输入	PMStrategyIndex：表示后置功耗管理策略的索引 0x00：芯片退出 standby 之后的功耗策略 其他：空操作 bakSFRBuf：表示存储 3 个字长的 SFR 备份值的地址，两个 SFR 依次是 SYSCLKCFG、SYSICSCFG
输出	无
返回值	无
资源使用	
说明	无

3.2.2.5 开启 ICS 功能函数

3.2.2.5.1 函数说明

本函数负责打开 ICS，使能 ICS 自适应。

仅用于双信开发使用

根据芯片功耗管理要求，在芯片一次上电期间，COS 或者使用 ICS 功能，或者不使用 ICS 功能；即在芯片一次上电期间，或者调用且仅调用一次本函数，或者从不调用本函数。

3.2.2.5.2 接口定义

函数接口	void HED_OpenICS(void)
输入	无
输出	无
返回值	无
资源使用	----
说明	（1）在芯片一次上电期间，COS 或者调用且仅调用一次本函数，或者从不调用本函数； （2）调用本函数的时机，射频接口下建议是在收到 1st APDU 之后，接触接口下建议是发送 ATR 后；

3.2.2.6 ICS 配置函数

3.2.2.6.1 函数说明

本函数用于实现软件设置 ICS 电流配置。

3.2.2.6.2 函数接口

函数接口	void HED_ChangeICSSet(unsigned char tgtICSSet)
输入	tgtICSSet:目标 ICSSET 值。对应的 ICS 电流值 ≈ (tgtICSSet +1) * 0.1mA
输出	无
返回值	无
说明	无

3.2.2.7 ICS 上限配置函数

3.2.2.7.1 函数说明

本函数用于实现软件设置 ICS 上限限制。

建议用户不要调用此函数进行配置。使用芯片默认的 ICS 配置即可。

3.2.2.7.2 函数接口

函数接口	void HED_ConfigICSLMT(unsigned int tgtICSLMT)
输入	tgtICSLMT：0x10~0x3F
输出	无

仅用于东信开发使用

返回值	无
说明	无

4 芯片掩膜 NVM API

4.1 概述

4.1.1 NVM 用户区分区介绍

本芯片用户 NVM 划分为 High Endurance Flash 区（简称 HEFlash）、Common Flash 和用户参数区三个区。其中算法区可以划最多 112K 到 Common Flash，并保证 Common Flash 和 HEFlash 前后连续。当 M1 不使能时，M1 的 16K 空间将被释放为 Common Flash，并可以使用 Common Flash 擦写相关 API 进行擦写。但是地址不和用户原有 Common Flash 连续，而是在 0x1FF00000 这个首地址上。

HEFlash 区用于存储 Code 和 Data；Common Flash 一般用于存储 Code；用户参数区用于存放用户可配置的芯片特定参数。

存储器读：可通过 CPU 指令直接读存储器。

存储器写：用户只能调用本章节描述的掩膜的擦写函数完成对用户 Flash 的擦写操作。

4.1.2 NVM API 的主要特征

4.1.2.1 支持 FTL 机制

HEFlash 支持 FTL（Flash Translation Lay）机制，可均衡 Sector 内每页 Flash 的擦写寿命。经过均衡处理后，擦写寿命可达到“每 Sector 内可以重复擦写 X 次以上”（X 值参见《芯片 SPEC》第 10 章节）。

如果一个 sector 内的擦写热点为 n 页，则这 n 页的平均擦写寿命为$\frac{X}{n}$次。

4.1.2.2 页内编程原子性

HEFlash 的大部分 API 支持“页内编程原子性”，可保障该页内的数据在擦写过程中发生断电后，数据不丢失。

【术语说明】

“页内编程原子性”：Atomic Page Program，通过调用 HEFlash 的 API 进行擦写，可保障该页内的数据在擦写过程中发生断电后，数据不丢失——要么为擦写前的旧数据，要么为预期写入的新数据。

4.1.3 Flash 增强稳定性功能

为减少外部干扰影响，Flash 提供增强稳定性功能，本节的 API 需要配合《HED_增强芯片稳定性方案》提及的操作方式使用。

仅用于东信开发使用

4.1.4 NVM API 注意事项

【注意 1】本芯片用户态下，用户 COS 程序无权限擦写 Flash 区（包括用户参数区与 HEFlash 区），必须调用本节提供的 API 执行 Flash 擦写操作。

【注意 2】NVM API 的函数执行期间均不响应各种中断，但期间产生的中断信号保留，在 NVM API 执行完后可立即响应。

【注意 3】本节和擦写相关的 API，只有在正确执行过“增强芯片稳定性方案类 API”后，才可以正确擦写 NVM。否则会执行当前程序的 Hardfault 中断处理程序。

【注意 4】当函数返回 0xFFBBAADD 时，表明之前在调用 HEFlash 擦写函数过程中出现过返回值 0xFFFFFFFF。因为之前在调用 HEFlash 已经出现错误，再擦写只会增加错误，需要通过上电复位尝试修复。如果复位后擦写函数依然报错，表明 Flash 损坏。

4.2 HEFlash 区擦写函数

HEFlash 区包括 n*124KB User Flash 区。其中 n 为 1 或者 2，代表有几个 sector。

4.2.1 初始化 HEFlash 区函数

4.2.1.1 功能介绍

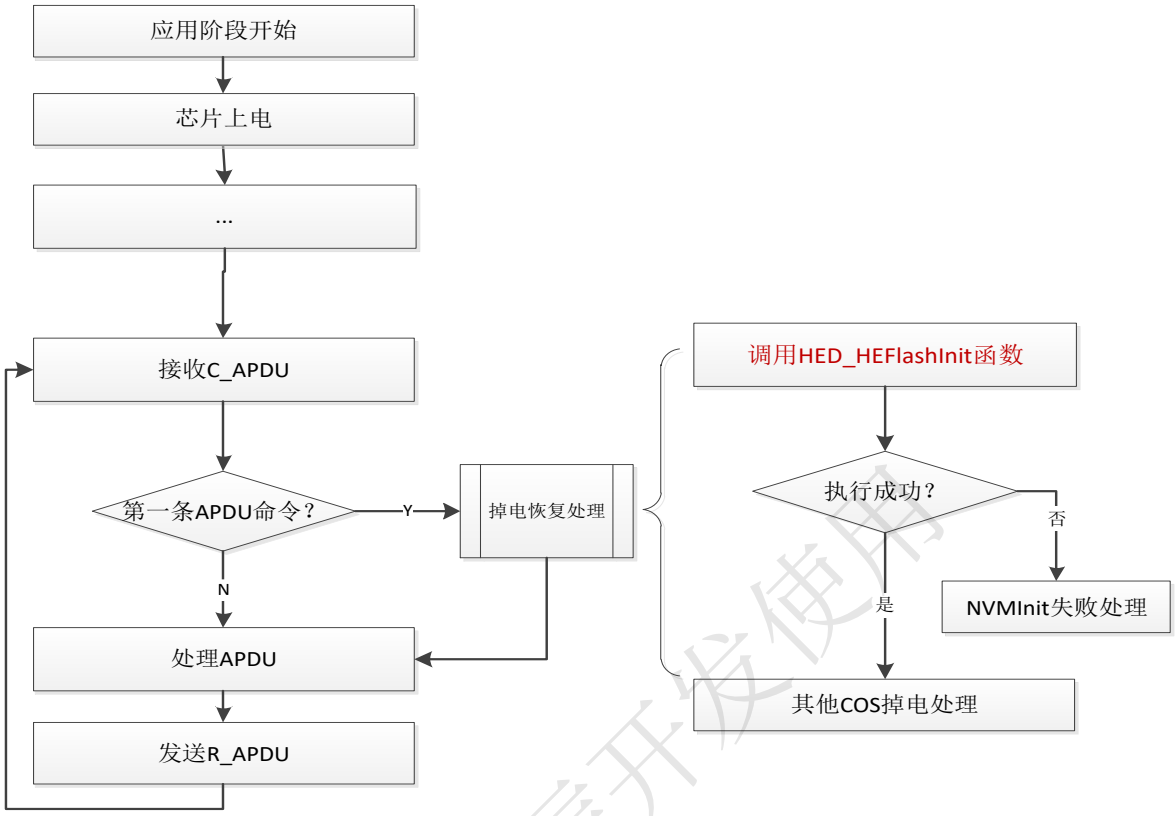
建立 HEFlash 运行状态。如果上次断电发生在擦写期间，则会本函数还会包含断电恢复。

【注意】用户必须在芯片工作后擦写 HEFlash 区前调用一次本函数，否则调用其他 HEFlash 擦写函数会返回 0xFFFF55AA 的错误值。必须在 COS 自身设计的断电恢复的处理时机，调用本函数。

函数被调用的时机： 用户 COS 在接收到第一条 APDU 后，先处理 HEFlash 的页断电恢复（通过调用 HED_HEFlash_Init），再处理 COS 自身的应用断电恢复，再处理该 APDU 的功能。

调用示例如下：

仅用于华虹开发使用



4.2.1.2 接口定义

函数接口	unsigned int HED_HEFlash_Init(void)
输入	无
输出	无
返回值	➢ 0x00000000：表示初始化成功； ➢ 0xFFFFFFFF：表示 HEFlash 失效； ➢ 0xFFFF55AA：表明 HEFlash 未创建。
资源使用	约 420 字节栈 RAM。
说明	➢ 注：函数性能请参考附录 NVM 擦写时间表。

4.2.2 HEFlash 擦写函数

4.2.2.1 功能介绍

本函数用于 HEFlash 区的页内擦写操作，每次完成**一页内的任意长度的擦写操作**。写一页和写一字节的时间相同。

【注意】本函数支持页内编程原子性。

本函数是 HEFlash 中最重要的的擦写函数，可满足应用中绝大多数擦写操作。

仅用于东信开发使用

4.2.2.2 接口定义

函数接口	unsigned int HED_HEFlash_EWPage(unsigned int FlashAddr, unsigned char *DataBuf, unsigned int len)
输入	➢ FlashAddr：目标地址； ➢ DataBuf：源地址，即指向待写入 FLASH 的数据。DataBuf 可支持 RAM 地址、支持 UserFlash 地址（但源 UserFlash 地址和目的 FlashAddr 不能有重叠部分）； ➢ len：dataBuf 有效数据字节长度［长度要求见说明 1］。
输出	无
返回值	➢ 0x00000000：表示擦写成功； ➢ 0xFFFFFFFF：表明 HEFlash 失效，即擦写失败； ➢ 0xFFFF55AA：表明 FlashAddr 或 DataBuf 地址错误、或 len 错误、或 HEFlash 未创建、或未执行 HED_HEFlash_Init 函数； ➢ 0xFFBBAADD：表明在之前调用 HEFlash 擦写、只擦函数过程中出现过返回值 0xFFFFFFFF，则本次上电期间禁止再次调用 HEFlash 擦写函数。
资源使用	约 420 字节栈 RAM
说明	➢ FlashAddr 和 len 组合所表示的地址范围应在一个 Flash 页内。例如，FlashAdd = 0x1000_01F0， len =0x11，这种组合的数据是跨页的，函数将报错； ➢ 擦写后读出比对，返回擦写成功与否的状态（擦写指将 FLASH 某块区域擦写成用户指定的数据）； ➢ 如果擦写不成功，表明 HEFlash 失效； ➢ API 判断 FlashAddr 参数的正确性。如果 FlashAddr 不在 HEFlash 范围内，将报错； ➢ 注：函数性能请参考附录 NVM 擦写时间表。

4.2.3 特殊场景高可靠性擦写函数

本章节函数可用于高级用户对应用 COS 的事物断电保护处理中的断电标志的相关的字编程处理。当然，8.2.2 节中的 HED_HEFlash_EWPage 函数也可以完成对 1 个字节的含原子性保护的擦写处理。在写的字节数较多时，HED_HEFlash_EWPage 函数的性能优于 HED_HEFlash_ErasePage+HED_HEFlash_WriteNWord，更优于 HED_HEFlash_ErasePage+HED_HEFlash_WriteWord 的组合，请根据需要组合使用。

4.2.3.1 HEFlash 页擦函数

4.2.3.1.1 功能介绍

本函数用于 HEFlash 区的页擦操作，每次完成逻辑页的页擦操作和读比对。可将该逻辑页擦为全 FF。

仅用于芯片开发使用

【注意】本函数支持页内编程原子性。即页擦动作受原子性保护。

4.2.3.1.2 接口定义

函数接口	unsigned int HED_HEFlash_ErasePage(unsigned int FlashAddr, unsigned int FlashAddrINV)
输入	➢ FlashAddr：目标页地址； ➢ FlashAddrINV：flashAddr 取反。
输出	无
返回值	➢ 0x00000000：表示擦写成功； ➢ 0xFFFFFFFF：表明 HEFlash 失效，即擦写失败； ➢ 0xFFFF55AA：表明 FlashAddr 地址错误、或 FlashAddrINV 错误、或 HEFlash 未创建、或未执行 HED_HEFlash_Init 函数； ➢ 0xFFBBAADD：表明在之前调用 HEFlash 擦写、只擦函数过程中出现过返回值 0xFFFFFFFF,则本次上电期间禁止再次调用 HEFlash 擦写函数。
资源使用	约 420 字节栈 RAM
说明	

4.2.3.2 HEFlash 字写函数

4.2.3.2.1 功能介绍

本函数用于 HEFlash 区的字写操作。每次完成逻辑地址的字写操作和读比对。用时为一次字写时间。要求被写空间在调用本函数之前为擦除后的状态。禁止对同一地址空间擦一次，写多次。

【注意】不支持被写字的原子性，即该被写字的字空间可能存在断电数据丢失的现象。

4.2.3.2.2 接口定义

函数接口	unsigned int HED_HEFlash_WriteWord(unsigned int FlashAddr, unsigned int value, unsigned int checkWord)
输入	➢ flashAddr：目标字地址； ➢ value：待写入的数据； ➢ checkWord：FlashAddr 和 value 的异或值。
输出	无
返回值	➢ 0x00000000：表示擦写成功； ➢ 0xFFFFFFFF：表明字写失败（可能是 HEFlash 寿命到了，或被写空间不是 FF）； ➢ 0xFFFF55AA：表明 FlashAddr 地址错误、或 checkWord 错误、或 HEFlash 未创建、或未执行 HED_HEFlash_Init 函数； ➢ 0xFFBBAADD：表明在之前调用 HEFlash 擦写、只擦函数过程中出现过返回值 0xFFFFFFFF,则本次上电期间禁止再次调用 HEFlash 擦写函数。
资源使用	----

仅用于系统开发使用

说明	

4.2.3.3 HEFlash 连续字写函数

4.2.3.3.1 功能介绍

本函数用于 HEFlash 区的连续写操作（不允许跨页）。每次完成逻辑地址的 N 字写操作和读比对。用时基本为 N 字写时间。要求被写空间在调用本函数之前存储的是擦后的 FF 状态。禁止对同一地址空间擦一次，写多次。

【注意】

1) **不支持被写字的原子性，即该被写字的字空间可能存在断电数据丢失的现象。**
2) **目标 FlashAddr 必须为字对齐。**
3) **被写 Flash 地址空间不可跨页，否则函数报错；**

4.2.3.3.2 接口定义

函数接口	unsigned int HED_HEFlash_WriteNWord(unsigned int FlashAddr, unsigned int *DataBuf, unsigned int len)
输入	➢ FlashAddr：目标字地址； ➢ DataBuf：指向 RAM 中待写入的数据（字对齐）； ➢ len：待写数据的 word 数量（0≤len≤0x80）。
输出	无
返回值	➢ 0x00000000：表示擦写成功； ➢ 0xFFFFFFFF：表明字写失败（可能是 HEFlash 寿命到了，或被写空间不是 FF）； ➢ 0xFFFF55AA：表明 FlashAddr 或 DataBuf 地址错误、或 len 错误、或 HEFlash 未创建、或未执行 HED_HEFlash_Init 函数，或 FlashAddr 地址不是字对齐； ➢ 0xFFBBAADD：表明在之前调用 HEFlash 擦写、只擦函数过程中出现过返回值 0xFFFFFFFF,则本次上电期间禁止再次调用 HEFlash 擦写函数。
资源使用	----
说明	➢ 被写 Flash 地址空间不可跨页。否则报 len 错误。

仅供华东师范大学使用

4.3 Common Flash 区擦写函数

4.3.1 功能介绍

本函数用于 **Common Flash 区**的页擦写操作，每次完成一整页的擦\写\校验处理。

【注意】不支持页内数据原子性。

不建议客户使用本函数去做应用中涉及的擦写操作。即客户应将 Common Flash 区作为程序区使用，仅在某些特定情况下擦写 CommonFlash 区。

4.3.2 接口定义

函数接口	unsigned int HED_CommonFlash_EWPage(unsigned int FlashAddr, unsigned int *DataBuf)
输入	➢ FlashAddr：目标 Page 的起始地址（必须为首地址）； ➢ DataBuf：源地址，即指向 RAM 中待写入 FLASH 的数据（指向数据的长度必须为 512 字节）。
输出	无
返回值	➢ 0x00000000：表示擦写成功； ➢ 0xFFFFFFFF：表明擦写失败； ➢ 0xFFFF55AA：FlashAddr 或 DataBuf 地址错误。
资源使用	----
说明	➢ 地址必须为该 Page 的首地址；如果输入地址不是 Page 首地址，则会从 Page 的首地址开始写入； ➢ 擦写后读出比对，返回擦写成功与否的状态（擦写指将 FLASH 某块区域擦写成用户指定的数据）； ➢ 函数内部会对地址进行合法性判断，如果非法，会返回 0xFFFF55AA 错误指示；

4.4 Common Flash 区页擦函数

4.4.1 功能介绍

本函数用于 **Common Flash 区**的页擦操作，每次完成一整页的擦\校验处理。

【注意】不支持页内数据原子性。

不建议客户使用本函数去做应用中涉及的擦操作。即客户应将 Common Flash 区作为程序区使用，仅在某些特定情况下擦 CommonFlash 区。

4.4.2 接口定义

函数接口	unsigned int HED_CommonFlash_ErasePage(unsigned int FlashAddr, unsigned int FlashAddrINV)

仅限与信华发使用

输入	➢ FlashAddr：目标页地址； ➢ FlashAddrINV：flashAddr 取反。
输出	无
返回值	➢ 0x00000000：表示擦写成功； ➢ 0xFFFFFFFF：表明擦写失败； ➢ 0xFFFF55AA：FlashAddr 或 FlashAddrINV 地址错误。
资源使用	----
说明	

4.5 Common Flash 区字写函数

4.5.1 功能介绍

本函数用于 **Common Flash 区**的字写操作，每次完成指定区域的写\校验处理。

【注意】不支持页内数据原子性。

不建议客户使用本函数去做应用中涉及的写操作。即客户应将 Common Flash 区作为程序区使用，仅在某些特定情况下擦 CommonFlash 区。

4.5.2 接口定义

函数接口	unsigned int HED_CommonFlash_WriteNWord(unsigned int FlashAddr, unsigned int *DataBuf, unsigned int len)
输入	➢ FlashAddr：目标字地址； ➢ DataBuf：指向 RAM 中待写入的数据（字对齐）； ➢ len：待写数据的 word 数量（0≤len≤0x80）。
输出	无
返回值	➢ 0x00000000：表示擦写成功； ➢ 0xFFFFFFFF：表明字写失败（可能是 HEFlash 寿命到了，或被写空间不是 FF）； ➢ 0xFFFF55AA：表明 FlashAddr 或 DataBuf 地址错误、或 len 错误，或 FlashAddr 地址不是字对齐；
资源使用	----
说明	

4.6 用户参数区 Flash 擦写函数

4.6.1 功能介绍

本函数用于用户参数区 Flash 的页擦写操作，每次完成一整页的擦\写\校验处理。

【注意】不支持页内数据原子性。

仅电子信息开发使用

用户参数区擦写功能主要用于用户修改用户参数区相关数据，例如修改卡片状态切换标识，实现退回下载态功能等。

用户参数区擦写用户状态切换标识、卡片状态切换标识和只读区配置约束如下：

1) 永久用户态下，用户状态切换标识不允许修改成暂时用户态和永久用户态时；
2) 永久用户态下，卡片状态切换标识不允许修改成下载态；
3) 永久用户态下，只读区配置不可改写成与 NVM 中不一样的值。

4.6.2 接口定义

函数接口	unsigned int HED_UserPara_EWPage(unsigned int FlashAddr, unsigned int *DataBuf)
输入	➢ FlashAddr：目标 Page 的起始地址（必须为首地址）； ➢ DataBuf：源地址，即指向 RAM 中待写入 NVM 的数据（指向数据的长度必须为 512 字节）。
输出	➢ 无
返回值	➢ 0x00000000：表示擦写成功； ➢ 0xFFFFFFFF：表明擦写失败； ➢ 0xFFFF55AA：FlashAddr 或 DataBuf 地址错误。
资源使用	----
说明	➢ 地址必须为该 Page 的首地址；如果输入地址不是 Page 首地址，则会从 Page 的首地址开始写入； ➢ 擦写后读出比对，返回擦写成功与否的状态（擦写指将 FLASH 某块区域擦写成用户指定的数据）； ➢ 函数内部会对地址进行合法性判断，如果非法，会返回 0xFFFF55AA 错误指示； ➢ 满足上述三个约束条件时，函数不启动擦写，直接返回 0xFFFF55AA 错误指示。

4.7 读 NVMAPI 版本函数

4.7.1 功能介绍

本函数用于读取 NVMAPI 版本。

4.7.2 接口定义

函数接口	unsigned int HED_NVMAPI_Version(void)
输入	无
输出	无
返回值	➢ 版本。
资源使用	----
说明	无

仅用于华大开发使用

5 附录 A 对称算法 Type 定义

对称算法 API 中都有工作模式选择参数 Type，对于 Type 的配置定义遵循如下规则：

b7	b6	b5	b4	b3	b2	b1	b0	说明
x	-	-	-	-	-	-	-	0 – 不支持 MAC 计算 1 – 支持 MAC 计算
-	x	-	-	-	-	-	-	0 – 加密 1 – 解密
-	-	x	x	x	-	-	-	加解密方式： 000 – ECB 010 – CFB 100 – CBC 110 – OFB 001 – CTR
-	-	-	-	-	x	-	-	Key 来源： 0 – 内部 1 – 外部
-	-	-	-	-	-	x	x	Key 类型： 00 – key1 01 – key2 10 – key3

注 1：

1) 在 DES 中 key1 对应单 DES，key2 对应 2keyTDES，key3 对应 3keyTDES；

2) 在 AES 中 key1 对应 128 位 AES，key2 对应 192 位 AES，key3 对应 256 位 AES。

注 2：“-” 表示该位的值不关心，缺省填 0。

6 附录 B NVM 擦写时间

NVM 操作方式	对应函数名	条件	典型值（us）
FTL 掉电恢复	HED_HENVM_Init	无掉电恢复 CPU@88MHZ	40
		有掉电恢复 CPU@88MHZ	1470
(页内)擦写	HED_HENVM_EWPage	CPU@88MHZ	3680
页擦	HED_HENVM_ErasePage	CPU@88MHZ	1530
字写	HED_HENVM_WriteWord	CPU@88MHZ	110
普通区(页内)擦写	HED_CommonNVM_EWPage	CPU@88MHZ	2700

仅用于东信开发使用

仅用于东信开发使用